



UNIVERSITÀ DEL SALENTO

Dipartimento di Ingegneria dell'Innovazione
Corso di Laurea Magistrale in Ingegneria Informatica

Progetto di Esame di INTERNET OF THINGS

Sistema intelligente di raccolta e annotazione dati biomedici

Docente:

Prof. Luigi Patrono

Studente:

Giuri Francesco

Anno Accademico 2025 - 2026

Indice

1	Introduzione	5
1.1	Contesto e Motivazione	5
1.2	Descrizione del Progetto	6
1.3	Obiettivi	6
1.4	Struttura della Relazione	7
2	Analisi dei Requisiti	8
2.1	Requisiti Funzionali	8
2.1.1	Acquisizione e Trasmissione Dati	8
2.1.2	Gestione dei Profili	8
2.1.3	Rilevamento delle Anomalie	9
2.1.4	Annotazione Clinica	9
2.1.5	Notifiche	9
2.2	Requisiti Non Funzionali	10
2.2.1	Prestazioni	10
2.2.2	Sicurezza e Privacy	10
2.2.3	Affidabilità	10
2.2.4	Manutenibilità e Scalabilità	10
2.3	Casi d’Uso Principali	10
3	Stato dell’Arte	12
3.1	Monitoraggio Remoto dello Scompenso Cardiaco	12
3.2	Lavori Correlati	13
4	Tecnologie Utilizzate	15
4.1	Protocolli di Comunicazione e Sicurezza	15
4.2	Broker MQTT: Eclipse Mosquitto	16
4.3	Back-end: Python e FastAPI	16
4.4	Modulo AI: Random Forest e Scikit-learn	16
4.5	App Paziente: Python e Kivy	17

4.6	Dashboard Medico	17
4.7	Containerizzazione: Docker e Docker Compose	17
4.8	Hardware di Acquisizione: Dongle e Prototipo IIT	18
4.8.1	Dongle nRF52840 MDK USB	18
4.8.2	Prototipo di Board Sensoristica (IIT)	18
5	Soluzione Proposta	20
5.1	Architettura del Sistema	20
5.1.1	Acquisizione: IIT BioDataAcq	21
5.1.2	Brokering ed Elaborazione	21
5.1.3	API	22
5.1.4	Persistenza	22
5.1.5	Validazione e Retraining	22
5.1.6	Comunicazione Real-Time	23
5.2	Pipeline di Acquisizione e Classificazione	23
5.2.1	Pubblicazione del Messaggio	23
5.2.2	Orchestrazione della Classificazione	24
5.2.3	Gestione della Finestra ECG Estesa	24
5.2.4	Notifica e Raggruppamento in Episodi	24
5.3	Moduli di Classificazione	25
5.3.1	ECGClassifier	25
5.3.2	PosturaClassifier	25
5.3.3	TemperaturaClassifier	26
5.4	Persistenza dei Dati	26
5.4.1	MongoDB: Annotazioni	27
5.4.2	MySQL: Profili Anagrafici	27
5.5	Ciclo di Validazione Clinica	28
5.5.1	Recupero degli Episodi	28
5.5.2	Applicazione della Validazione	28
5.5.3	Storico e Rivalutazione	29
5.6	Retraining Periodico	29
5.6.1	Schedulazione	29
5.6.2	Estrazione del Dataset ed Addestramento	29
5.6.3	Sostituzione Atomica e Hot-Reload	30
5.7	REST API e Autenticazione	31
5.7.1	Autenticazione	31
5.7.2	Endpoint Principali	31
5.8	Dashboard Medico — React	32

5.8.1	Doppio canale di aggiornamento: polling REST e WebSocket MQTT	32
5.8.2	Connessione WebSocket — <code>useMqtt</code>	33
5.8.3	Debounce degli allarmi	33
5.8.4	Gestione della sessione e scadenza token — <code>apiFetch</code>	34
5.8.5	Modal di validazione unificato	34
5.8.6	Traccia ECG estesa — costruzione asincrona lato client	35
5.8.7	Notifiche desktop e feedback sonoro	35
5.9	Integrazione App Paziente	36
5.9.1	Sessione paziente — <code>patient_session.py</code>	36
5.9.2	Login paziente — <code>patient_login.py</code>	36
5.9.3	Bridge MQTT — <code>mqtt_bridge.py</code>	36
5.9.4	Notifiche e storico anomalie — <code>patient_anomalies.py</code>	37
5.9.5	Considerazioni conclusive sull'integrazione	38
5.10	Considerazioni di Sicurezza	38
5.10.1	TLS end-to-end	39
5.10.2	Isolamento dei dati per paziente in processi condivisi	39
5.10.3	Gestione degli errori in scenari concorrenti e di rete instabile	40
6	Validazione Funzionale e Prestazionale	42
6.1	Setup di Test	42
6.2	Validazione Funzionale End-to-End	43
6.2.1	Percorso Testato	43
6.2.2	Esito dei Test	46
6.3	Prestazioni dei Modelli di Machine Learning	47
6.3.1	ECGClassifier — <code>chfdb</code>	47
6.3.2	PosturaClassifier — <code>MHEALTH</code>	48
6.3.3	Discussione dei Risultati	48
6.4	Osservazioni su Latenza e Affidabilità	49
6.4.1	Latenza tra Evento Anomalo e Notifica	49
6.4.2	Osservazioni Pratiche col Dispositivo Reale	50
7	Conclusioni e Sviluppi Futuri	52
7.1	Sintesi del Lavoro Svolto	52
7.2	Limiti dell'Implementazione Attuale	53
7.2.1	Modelli di Machine Learning	53
7.2.2	Architettura e Gestione dello Stato	53
7.2.3	Sicurezza e Portabilità	54
7.3	Sviluppi Futuri	54
7.3.1	Estensione dei Parametri Fisiologici Monitorati	54

7.3.2	Interoperabilità Clinica: HL7 FHIR	55
7.3.3	Robustezza della Connettività Lato Paziente	55
7.3.4	Gestione del Ciclo di Vita dello Stato per Paziente	55
7.3.5	Hardening della Sicurezza per il Deployment in Produzione . . .	55
Riferimenti Bibliografici		57

Capitolo 1

Introduzione

1.1 Contesto e Motivazione

Lo *scompenso cardiaco* (Heart Failure, HF) è una sindrome clinica cronica che si manifesta quando il cuore non riesce più ad assolvere la propria funzione di pompa, venendo meno alla sua capacità di garantire un adeguato apporto di sangue — e quindi di ossigeno e nutrienti — a tutti i tessuti dell’organismo [1]. Con una prevalenza stimata di oltre 64 milioni di persone a livello mondiale [2], rappresenta una delle principali cause di ricovero ospedaliero e mortalità nei Paesi industrializzati. In Italia, la frequenza della patologia si attesta intorno al 2% della popolazione generale, ma cresce sensibilmente con l’avanzare dell’età, raggiungendo circa il 15% in entrambi i sessi oltre gli 85 anni [1].

Sul piano clinico, lo scompenso si distingue in forma **sistolica**, dovuta a un’inefficace funzione contrattile, e **diastolica**, legata a un alterato riempimento ventricolare. La valutazione della *frazione d’iezione* del ventricolo sinistro, generalmente tramite ecocardiogramma, permette di classificare i pazienti e guidare la terapia. I sintomi principali comprendono dispnea da sforzo, dispnea notturna in posizione supina, edema periferico e marcata astenia [1]. La diagnosi precoce è spesso difficile a causa dell’aspecificità e della variabilità dei sintomi nel tempo.

È proprio in questo scenario che il paradigma dell’Internet of Medical Things (**IoMT**) offre un’opportunità concreta per trasferire parte di questo monitoraggio dall’ospedale al domicilio del paziente. Sensori indossabili a bassa potenza, protocolli di comunicazione senza fili efficienti (BLE, MQTT) e tecniche di intelligenza artificiale (AI) permettono di raccogliere, trasmettere e analizzare flussi di dati biomedici in modo continuo, con latenze nell’ordine dei secondi, contribuendo così a intercettare precocemente le fasi di riacutizzazione.

1.2 Descrizione del Progetto

Il progetto “*Sistema Intelligente di Raccolta e Annotazione Dati Biomedici*” si propone di realizzare un’architettura IoT **closed-loop** per il monitoraggio domiciliare di pazienti con HF. Il sistema è progettato attorno a tre componenti principali:

1. **Data Acquisition Layer:** dispositivo wearable indossato dal paziente, dotato di sensori multiparametrici (ECG, movimento e postura, temperatura corporea), i cui dati vengono raccolti da un’applicazione di acquisizione e inoltrati verso il back-end di elaborazione secondo un paradigma publish-subscribe.
2. **Notification Layer:** sistema di allarme che, in caso di anomalie cliniche rilevate, notifica tempestivamente il medico e il paziente coinvolto.
3. **AI & Annotation Layer:** modulo di classificazione automatica dei segnali fisiologici, integrato con un sistema di annotazione che raccoglie sia gli esiti automatici sia le validazioni cliniche del medico, alimentando un ciclo di miglioramento periodico dei modelli.

1.3 Obiettivi

Il progetto si propone di realizzare un sistema funzionante e testato su scenari reali, articolato attorno ai seguenti obiettivi:

- Acquisire in continuo parametri fisiologici multimodali — battito cardiaco, temperatura corporea, movimento e postura — da sensori indossabili, integrandosi con un’applicazione di acquisizione.
- Rilevare anomalie clinicamente rilevanti in tempo reale attraverso un modulo di classificazione automatica lato server, minimizzando la latenza tra l’evento fisiologico e la notifica al medico.
- Arricchire i dati grezzi con un sistema di annotazione che integra il giudizio automatico dei classificatori e la validazione clinica del medico, alimentando un ciclo di miglioramento periodico che chiude il loop tra IA e giudizio clinico.
- Validare l’intero sistema attraverso test end-to-end, sia con dati simulati sia con acquisizione reale dal dispositivo wearable, verificando il flusso completo fino alla generazione delle annotazioni e delle notifiche.

1.4 Struttura della Relazione

La relazione è organizzata in sette capitoli, ciascuno dei quali affronta una fase specifica del percorso che va dall'analisi del problema alla validazione della soluzione realizzata.

Il **Capitolo 2** definisce i requisiti funzionali e non funzionali del sistema, individuando i casi d'uso principali e i vincoli di sicurezza, prestazione e conformità normativa che hanno guidato le scelte progettuali.

Il **Capitolo 3** contestualizza il lavoro nell'ambito della letteratura scientifica e tecnologica, esaminando le soluzioni esistenti per il monitoraggio remoto dello scompenso cardiaco, i protocolli IoT più diffusi in ambito eHealth e le tecniche di anomaly detection applicate ai segnali biomedici.

Il **Capitolo 4** illustra le tecnologie e gli strumenti adottati, motivando le scelte effettuate in termini di linguaggi, framework, protocolli di comunicazione e database.

Il **Capitolo 5** presenta la soluzione proposta nella sua interezza, descrivendo l'architettura del sistema, il flusso dei dati tra i componenti e le interfacce API progettate per l'interazione tra gateway, back-end e dashboard medico.

Il **Capitolo 6** riporta la validazione funzionale e prestazionale del sistema, documentando i test end-to-end condotti con sensori reali e analizzando i risultati ottenuti in termini di latenza, correttezza delle annotazioni e affidabilità del Notification Layer.

Il **Capitolo 7** conclude il lavoro con una sintesi critica dei risultati raggiunti, evidenziando i limiti dell'implementazione attuale e delineando i possibili sviluppi futuri, tra cui l'estensione a ulteriori parametri fisiologici e l'integrazione con standard di interoperabilità clinica come HL7 FHIR [3].

Capitolo 2

Analisi dei Requisiti

2.1 Requisiti Funzionali

I requisiti funzionali descrivono le funzionalità che il sistema deve obbligatoriamente fornire. Sono organizzati in cinque aree tematiche: acquisizione e trasmissione dei dati, gestione dei profili, rilevamento delle anomalie, annotazione clinica e notifiche.

2.1.1 Acquisizione e Trasmissione Dati

Il sistema deve raccogliere in tempo reale tre tipologie di segnali fisiologici dal paziente: l'attività cardiaca, tramite gli intervalli R-R derivati dal segnale ECG; la temperatura corporea; il movimento e la postura.

I dati devono essere trasmessi dal dispositivo indossabile verso il sistema di elaborazione centrale con latenza minima. Tutte le comunicazioni di rete, in ogni tratta, devono essere cifrate, garantendo la riservatezza dei dati sanitari trattati.

2.1.2 Gestione dei Profili

Il sistema deve supportare due tipologie di utenti, medico e paziente, con permessi differenziati. Devono essere gestiti i profili anagrafici essenziali (nome, cognome) e la relazione di cura tra medico e paziente. Il paziente deve poter accedere al sistema in autonomia, senza credenziali tradizionali, tramite un codice di accesso univoco generato dal medico al momento della presa in carico.

2.1.3 Rilevamento delle Anomalie

Il sistema deve classificare automaticamente, in tempo reale e senza intervento umano, i segnali ricevuti da ciascun paziente, distinguendo condizioni fisiologiche normali da condizioni cliniche anomale. La classificazione deve avvenire separatamente per ciascun tipo di segnale (cardiaco, di movimento, di temperatura), in modo che un'anomalia rilevata su un segnale non venga mascherata o alterata dagli altri.

Per il segnale cardiaco, il sistema deve fornire, oltre all'esito normale/anomalo, un livello di confidenza della classificazione. Per il movimento, il sistema deve essere in grado di distinguere diverse tipologie di attività motoria, dal riposo ad attività fisiche intense. Il sistema deve inoltre poter essere riaddestrato periodicamente sulla base delle validazioni cliniche accumulate, così da migliorare nel tempo l'accuratezza della classificazione, senza richiedere interruzioni del servizio durante l'aggiornamento del modello.

2.1.4 Annotazione Clinica

Ogni anomalia rilevata deve generare un'annotazione persistente, recuperabile successivamente, contenente almeno il momento di rilevazione, il tipo di anomalia e il livello di confidenza. Anomalie riferite allo stesso evento clinico e ravvicinate nel tempo devono essere raggruppate in un unico episodio da sottoporre a validazione, evitando di presentare al medico più segnalazioni ridondanti per lo stesso evento.

Il medico deve poter validare ciascun episodio, classificandolo come evento reale o falso allarme, ed eventualmente corredarlo di note cliniche testuali. Le annotazioni devono essere consultabili sia dal medico che dal paziente a cui si riferiscono, con la possibilità di filtrarle per intervallo temporale e stato di validazione.

2.1.5 Notifiche

In presenza di un'anomalia critica, il sistema deve notificare tempestivamente sia il medico responsabile sia il paziente coinvolto. La notifica al medico deve essere visibile senza necessità di aggiornamento manuale della pagina ed essere accompagnata da un segnale distinguibile, mentre il paziente deve poter visualizzare in autonomia l'esistenza di un'anomalia recente relativa al proprio stato. Il sistema deve inoltre fornire una visione d'insieme aggiornata in tempo reale dei pazienti monitorati e degli episodi in attesa di validazione.

2.2 Requisiti Non Funzionali

2.2.1 Prestazioni

Il sistema è progettato per operare in tempo reale, garantendo una latenza contenuta tra l'acquisizione del dato e la sua elaborazione lato server. Il back-end deve inoltre poter gestire più pazienti monitorati contemporaneamente senza degradazione delle prestazioni né interferenza tra i rispettivi flussi di dati.

2.2.2 Sicurezza e Privacy

Trattandosi di dati sanitari, la sicurezza dei canali di comunicazione è un requisito non negoziabile: ogni scambio di dati, in ogni tratta del sistema, deve avvenire su canale cifrato. L'accesso alle funzionalità esposte dal back-end deve essere riservato a utenti autenticati — medici e pazienti registrati. Il trattamento dei dati deve essere conforme al Regolamento GDPR (EU 2016/679); in prospettiva di un eventuale utilizzo clinico reale, il sistema dovrebbe inoltre rispettare le norme MDR (EU 2017/745) per i dispositivi medici.

2.2.3 Affidabilità

Il sistema deve garantire un funzionamento continuo, dato che un'interruzione del monitoraggio potrebbe avere conseguenze cliniche. Il ri-addestramento periodico dei modelli, in particolare, non deve comportare interruzioni del servizio di classificazione in corso.

2.2.4 Manutenibilità e Scalabilità

L'architettura deve essere progettata in modo da poter accogliere nuove tipologie di sensore o nuovi classificatori in futuro, senza richiedere modifiche strutturali al core del sistema.

2.3 Casi d'Uso Principali

I principali casi d'uso del sistema sono illustrati nella tabella seguente. Per ciascuno di essi vengono specificati l'attore coinvolto, le precondizioni necessarie e il flusso principale delle interazioni previste.

Caso d'uso	Attore	Precondizione	Flusso principale
Monitoraggio continuo	Paziente	Sensori indossati e app di acquisizione attiva	L'app di acquisizione pubblica i dati sul broker MQTT. Il sistema classifica in tempo reale ciascun segnale e archivia gli esiti su MongoDB.
Rilevamento anomalia	Sistema	Monitoraggio attivo	Un classificatore rileva una finestra anomala, genera un'annotazione automatica — raggruppata con eventuali anomalie contigue in un unico episodio — e notifica in tempo reale medico e paziente.
Validazione medica	Medico	Allarme ricevuto sulla dashboard	Il medico visualizza l'episodio con la relativa traccia ECG e lo classifica come vero positivo o falso allarme, con note cliniche opzionali.
Consultazione storico	Paziente Medico	/ Login effettuato	L'utente consulta lo storico delle anomalie del paziente.
Creazione profilo	Medico	Sessione dashboard attiva	Il medico crea il profilo anagrafico del paziente su MySQL tramite la dashboard. Il sistema genera un codice univoco che il paziente usa per accedere all'app.
Ri-addestramento modello	Sistema	Nuove validazioni cliniche accumulate	Il sistema ri-addestra periodicamente il classificatore ECG combinando il dataset di base con le annotazioni validate dai medici, e sostituisce il modello in produzione senza interruzione del servizio.

Tabella 2.1: Casi d'uso principali del sistema.

Capitolo 3

Stato dell'Arte

3.1 Monitoraggio Remoto dello Scompenso Cardiaco

Il monitoraggio remoto dei pazienti con scompenso cardiaco (Remote Patient Monitoring, RPM) rappresenta uno dei filoni di ricerca più attivi nell'ambito della digital health. L'obiettivo principale è intercettare precocemente i segnali di deterioramento clinico, riducendo il ricorso a ricoveri ospedalieri d'urgenza e migliorando la qualità di vita del paziente.

Le soluzioni disponibili spaziano da approcci invasivi, basati su sensori impiantabili per la misurazione continua di parametri emodinamici, a soluzioni completamente non invasive basate su dispositivi indossabili — smartwatch, patch ECG, bracciali pressori — che consentono un monitoraggio continuo senza richiedere procedure interventistiche [4].

Una recente revisione sistematica della letteratura condotta da Rucco et al. [5] conferma la crescente integrazione di tecnologie IoT e intelligenza artificiale nel monitoraggio dello scompenso cardiaco, evidenziando al contempo la mancanza di sistemi che combinino acquisizione multimodale, rilevamento automatico delle anomalie e annotazione clinica strutturata — lacune che il sistema proposto in questo progetto si propone di colmare.

3.2 Lavori Correlati

La letteratura recente presenta diversi sistemi IoT per il monitoraggio remoto della salute che condividono obiettivi parzialmente sovrapponibili con il presente progetto. Un'analisi critica di questi contributi consente di collocare la soluzione proposta nel panorama esistente e di evidenziarne gli elementi distintivi.

Vithyalakshmi et al. [6] propongono un sistema indossabile basato su IoT per il monitoraggio continuo di parametri vitali — frequenza cardiaca, pressione arteriosa, temperatura corporea e saturazione dell'ossigeno — con rilevamento delle anomalie affidato a un modello di intelligenza artificiale a due livelli. Il sistema è stato valutato su scenari clinici generici, riportando un'accuratezza complessiva del 92,8% nella classificazione degli eventi e una latenza media di allerta pari a 12,3 secondi. Rispetto a questo lavoro, il sistema qui proposto si differenzia per la specializzazione esclusiva allo scompenso cardiaco, per l'adozione di un modello Random Forest addestrato su dati ECG reali e per l'integrazione di un meccanismo di annotazione clinica che consente al medico di validare gli allarmi come vero positivo o falso allarme, riducendo nel tempo il tasso di falsi positivi tramite il ciclo di retraining periodico del classificatore.

Naveed et al. [7] descrivono un sistema IoT per la rilevazione in tempo reale di eventi cardiaci acuti, con particolare attenzione all'infarto del miocardio. I dati raccolti da sensori indossabili — frequenza cardiaca, livello di stress e variazioni posturali — vengono trasmessi a un server IoT e analizzati tramite Support Vector Regression e reti neurali artificiali. Il contributo si distingue per l'impiego congiunto di due architetture di apprendimento automatico e per la capacità di rilevare variazioni nel livello di stress come segnale predittivo. Tuttavia, il sistema è orientato alla rilevazione di eventi acuti e non al monitoraggio longitudinale dello scompenso cardiaco cronico. Il presente lavoro affronta invece la gestione continua nel tempo di una patologia cronica, prevedendo un flusso dati persistente, una base di dati strutturata per le serie temporali e un ciclo di retroazione tra medico e sistema attraverso le annotazioni.

Tiwari et al. [8] presentano una rassegna applicativa dei dispositivi IoT indossabili per il monitoraggio della salute, impiegando sensori ECG, PPG, accelerometro e temperatura. Il lavoro esplora l'utilizzo di modelli di deep learning e di algoritmi di rilevamento delle anomalie in architetture che sfruttano sia il cloud computing sia l'edge computing per ridurre la latenza e ottimizzare l'utilizzo della banda. Gli autori discutono inoltre le problematiche di sicurezza, autonomia energetica e integrazione con i servizi sanitari. Sebbene questo contributo offra una panoramica tecnologica ampia e aggiornata, non presenta un sistema completo e integrato né una validazione sperimentale su una coorte di pazienti specifica. Il sistema proposto in questa tesi si distingue per l'implementazione concreta di un'architettura end-to-end — dalla comunicazione BLE

con i sensori fino alla dashboard del medico — con scelte progettuali motivate dalla patologia target e da requisiti di sicurezza espliciti, tra cui la cifratura TLS sul broker MQTT e l'autenticazione tramite JWT.

Hinrichs et al. [9] propongono un modello di machine learning per la predizione a breve termine del rischio di ospedalizzazione per scompenso cardiaco, sviluppato nell'ambito del trial randomizzato TIM-HF2 su dati di monitoraggio non invasivo trasmessi quotidianamente dai pazienti. Il modello supera significativamente l'algoritmo euristico convenzionale adottato nel trial e mostra una capacità di rilevazione precoce nelle settimane precedenti un evento di ospedalizzazione. Questo lavoro rappresenta il contributo clinicamente più affine al sistema proposto, poiché condivide la specializzazione nello scompenso cardiaco e l'impiego del machine learning su dati di telemonitoraggio reale. La differenza fondamentale risiede nel contesto applicativo: Hinrichs et al. operano in uno scenario di telemedicina istituzionale con trasmissioni periodiche supervisionate, mentre il sistema qui sviluppato è concepito per un monitoraggio domiciliare continuo e autonomo, arricchito da un meccanismo di annotazione clinica strutturata assente nel lavoro citato.

Capitolo 4

Tecnologie Utilizzate

4.1 Protocolli di Comunicazione e Sicurezza

La comunicazione tra i componenti del sistema si articola su due livelli distinti. A livello locale, il protocollo **BLE/USB** gestisce la trasmissione dei dati grezzi dal dispositivo wearable all'applicazione di acquisizione (*IIT BioDataAcq*), che funge da gateway verso il resto del sistema. A livello di rete, il protocollo **MQTT** (*Message Queuing Telemetry Transport*) è adottato per la comunicazione tra il gateway e il broker (Mosquitto), grazie alla sua leggerezza, al modello publish-subscribe e alla capacità di operare su connessioni instabili [10]. Lo stesso broker espone anche un listener WebSocket, tramite cui la dashboard medico riceve gli allarmi in tempo reale direttamente dal browser, senza necessità di un client MQTT nativo.

Gli endpoint REST esposti da FastAPI sono utilizzati per la gestione dei profili anagrafici, l'autenticazione e il recupero dello storico delle anomalie, sia da parte della dashboard medico che dell'applicazione paziente: quest'ultima li utilizza per il login tramite codice di accesso e per consultare periodicamente gli episodi relativi al proprio stato, mentre la dashboard li sfrutta per la gestione dei profili, la visualizzazione dei dati e la validazione clinica degli episodi.

La sicurezza dei canali è garantita tramite **TLS** (Transport Layer Security), applicato sia al protocollo MQTT (`mqtts://`) e al suo listener WebSocket (`wss://`), sia alle API REST (`https://`). I certificati, generati localmente tramite **mkcert** [11], evitano gli avvisi di sicurezza tipici dei certificati self-signed generati manualmente, al costo di una CA di fiducia solo sulla macchina su cui è installata. L'autenticazione dei medici avviene tramite **JWT**, con token firmato rilasciato al login e incluso in ogni richiesta successiva; l'accesso lato paziente avviene invece tramite un codice alfanumerico univoco, verificato dal server ad ogni richiesta.

4.2 Broker MQTT: Eclipse Mosquitto

Il broker MQTT adottato è **Eclipse Mosquitto** [12], una soluzione open source leggera e ampiamente diffusa in ambito IoT. Nel contesto di questo progetto, Mosquitto è containerizzato tramite Docker Compose e funge da punto centrale di smistamento dei messaggi tra l'app paziente IIT BioDataAcq, il modulo di classificazione IA e la dashboard medico, quest'ultima sottoscritta agli allarmi tramite WebSocket.

4.3 Back-end: Python e FastAPI

Il back-end del sistema è sviluppato in **Python**, linguaggio che offre un ecosistema maturo per lo sviluppo di API, l'elaborazione di segnali biomedici e il machine learning. Il framework adottato per la realizzazione delle API REST è **FastAPI** [13], basato su un'architettura asincrona nativa (`asyncio`/ASGI) che lo rende particolarmente adatto a scenari di elaborazione dati in tempo reale.

Il back-end è articolato in due processi Python distinti che operano in parallelo: il primo sottoscrive i topic MQTT, riceve le finestre di dati in ingresso, esegue l'inferenza AI e archivia automaticamente i risultati su MongoDB; il secondo espone gli endpoint REST per la gestione dei profili, per la validazione clinica delle anomalie da parte dei medici e per la comunicazione con la dashboard medico e con l'app paziente.

4.4 Modulo AI: Random Forest e Scikit-learn

Il modulo di anomaly detection si basa su due classificatori distinti, entrambi **Random Forest** [14] scelti per la loro robustezza su dataset sbilanciati, la semplicità di addestramento e l'interpretabilità dei risultati, ma addestrati su segnali e dataset differenti. Il primo, **ECGClassifier**, è addestrato in modalità supervisionata sugli intervalli R-R estratti dal *BIDMC Congestive Heart Failure Database* [15, 16], che fornisce registrazioni di pazienti reali con insufficienza cardiaca congestizia con annotazioni cliniche che distinguono le fasi stabili dalle fasi di crisi. Il secondo, **PosturaClassifier**, è addestrato su feature statistiche congiunte di accelerometro e giroscopio (6 assi) del sensore da polso/braccio, a partire dal dataset *MHEALTH* [17].

L'implementazione è basata su **Scikit-learn** [18], la principale libreria Python per il machine learning classico, che offre un'interfaccia uniforme per l'addestramento, la serializzazione e l'inferenza del modello. La lettura del dataset [15] è gestita tramite la libreria **wfdb** [19], che consente di accedere ai record del database direttamente in Python senza scaricare l'intero archivio.

4.5 App Paziente: Python e Kivy

L'applicazione lato paziente, *IIT BioDataAcq*, è sviluppata in **Python** con il framework **Kivy** [20], scelto dal team IIT per la sua natura cross-platform e per l'accesso diretto alle librerie Python di elaborazione del segnale utilizzate nella componente di acquisizione. Su questa applicazione si innesta il layer di integrazione MQTT, che pubblica periodicamente i campioni acquisiti — ECG, accelerazione e velocità angolare del sensore IMU, temperatura corporea — verso il broker, senza richiedere modifiche al core di acquisizione originale.

Il layer di integrazione gestisce inoltre il login del paziente tramite codice di accesso alfanumerico e la consultazione periodica, via API REST, dello storico degli episodi relativi al proprio stato clinico, notificati al paziente tramite badge e popup all'interno dell'applicazione stessa.

4.6 Dashboard Medico

La dashboard medico, tramite cui il cardiologo valida gli episodi anomali e gestisce i pazienti, è sviluppata in **React** [21] con il bundler **Vite** [22]. La comunicazione con il backend avviene su due canali: endpoint REST di FastAPI (per operazioni sincrone via **Fetch API**) e protocollo MQTT via WebSocket cifrata per gli allarmi in tempo reale, utilizzando il client **MQTT.js** [23].

Le notifiche al medico sono erogate tramite le **Web Notifications API** native del browser e un allarme sonoro via **Web Audio API**, migliorando la percezione di urgenza clinica. La dashboard risiede in un repository separato e utilizza gli stessi certificati TLS (mkcert) del backend per la connessione cifrata.

4.7 Containerizzazione: Docker e Docker Compose

Il deployment del back-end è gestito tramite **Docker** [24] e **Docker Compose**, che permettono di avviare con un unico comando tutti i servizi necessari — FastAPI, Mosquitto, MongoDB e MySQL — in ambienti containerizzati e isolati. Questa scelta rende il sistema riproducibile indipendentemente dall'ambiente di esecuzione, semplificando sia lo sviluppo che la fase di test e dimostrazione.

4.8 Hardware di Acquisizione: Dongle e Prototipo IIT

Il sistema acquisisce segnali fisiologici tramite un dispositivo composito che integra due componenti hardware distinti:

4.8.1 Dongle nRF52840 MDK USB

Il *dongle nRF52840 MDK USB* [25] è una piattaforma hardware open-source sviluppata da makerdiary attorno al System-on-Chip multiprotocollo **nRF52840** di Nordic Semiconductor. Il dongle integra un processore **ARM Cortex-M4F** ottimizzato per il basso consumo energetico, un controller USB 2.0 on-chip e supporto nativo per Bluetooth 5, Bluetooth Mesh, Thread, Zigbee, IEEE 802.15.4 e protocolli proprietari a 2.4 GHz, con antenna chip integrata. Il fattore di forma USB e il bootloader UF2 preinstallato ne semplificano la programmazione e l'integrazione come ponte di comunicazione tra il prototipo di sensori e l'applicazione di acquisizione *IIT BioDataAcq* in esecuzione su PC.

4.8.2 Prototipo di Board Sensoristica (IIT)

Connesso al dongle nRF52840 tramite interfaccia BLE troviamo il prototipo di board sensoristica sviluppato dall'Istituto Italiano di Tecnologia. La board integra sensori fisici — per l'acquisizione dell'attività cardiaca (ECG), del movimento e della postura tramite unità inerziale a 6 assi (accelerometro e giroscopio) e della temperatura corporea — organizzati in circuiti e moduli specializzati.

La board segue un'architettura modulare a schede intercambiabili, sviluppata attorno a un System-on-Chip centrale a duplice core **Nordic nRF5340** (due core ARM Cortex-M33, rispettivamente fino a 128 MHz e 64 MHz, con radio BLE 5.4), che gestisce controllo, alimentazione e connettività, mentre ciascuna modalità di sensing è realizzata su una scheda dedicata, elettricamente e meccanicamente indipendente, alimentata dalle stesse linee condivise a 3,0 V e 1,8 V e interconnessa alle altre tramite cavi flessibili (FFC). Questo approccio consente di comporre configurazioni hardware diverse a partire dallo stesso core, includendo solo i moduli sensoristici richiesti dall'applicazione specifica.

Nella configurazione utilizzata in questo progetto sono presenti tre moduli sensoristici: un front-end ECG a singola derivazione bipolare basato sul chip **MAX30003**, un'unità inerziale a 6 assi **LSM6DS3TR-C** (accelerometro e giroscopio, interfaccia SPI) e un sensore di temperatura digitale **TMP117** (risoluzione 0,0078 °C, accuratezza $\pm 0,1$ °C, interfaccia I²C). I dati acquisiti da ciascun modulo vengono trasmessi via BLE al don-

gle nRF52840 secondo un protocollo a pacchetti proprietario, in cui un identificativo di flusso a 4 bit distingue il modulo sensoristico di provenienza dai restanti campi del payload (conteggio campioni, timestamp a 16 bit), permettendo al gateway e all'applicazione di acquisizione di instradare correttamente i dati di ciascuna modalità senza conoscere a priori quali moduli siano fisicamente presenti sulla board.

Lo stato attuale della board prototipale rappresenta una fase transitoria verso il deployment clinico. L'architettura è concepita in modo che i sensori possano essere integrati, in una fase successiva, in un dispositivo wearable indossabile — ad esempio una piccola patch adesiva.

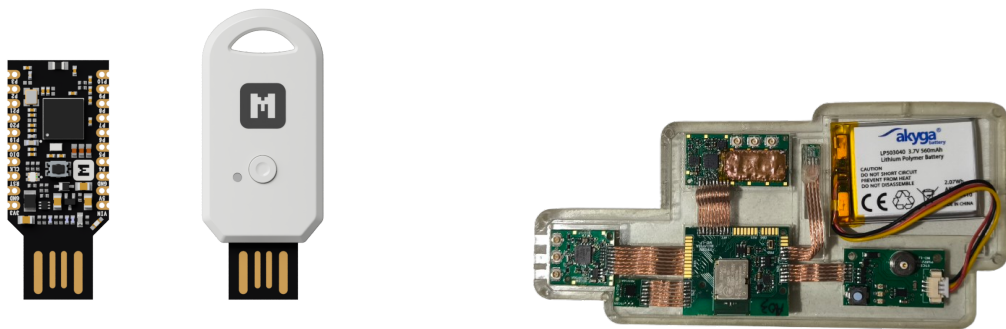


Figura 4.1: A sinistra: dongle nRF52840 MDK USB. A destra: prototipo di board sensoristica IIT con sensori ECG, IMU a 6 assi e temperatura integrati. Le due componenti comunicano via BLE.

Capitolo 5

Soluzione Proposta

5.1 Architettura del Sistema

L'architettura del sistema SmartCare è organizzata in tre strati logici, ciascuno con responsabilità ben definita: acquisizione e integrazione, elaborazione e persistenza, e infine feedback e retraining. Questi strati comunicano tramite protocolli asincroni (MQTT per il flusso di dati real-time) e sincroni (REST per le query cliniche), garantendo che il sistema rimanga reattivo anche in caso di latenze temporanee.

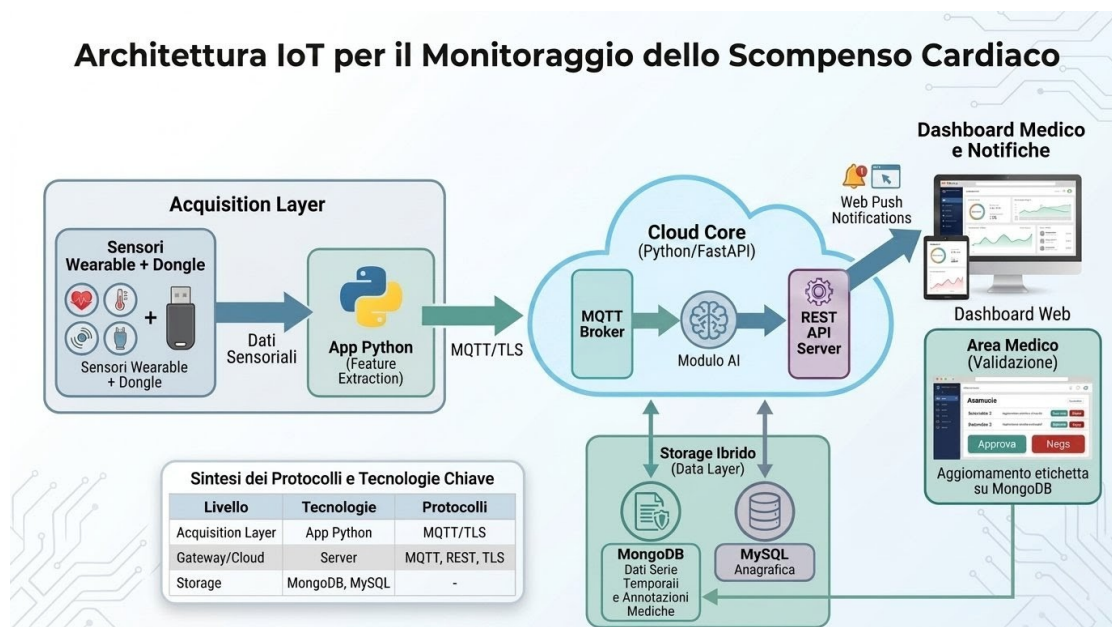


Figura 5.1: Schema architetturale end-to-end di SmartCare: dal dispositivo wearable (IIT BioDataAcq) tramite MQTT al broker Mosquitto, classificazione parallela (ECG, Postura, Temperatura), API REST, persistenza su MongoDB e MySQL, dashboard medico in React via WebSocket, e ciclo di retraining notturno.

5.1.1 Acquisizione: IIT BioDataAcq

Lo strato più alto è rappresentato dall'applicazione di acquisizione *IIT BioDataAcq*, che rimane il componente core per la raccolta dei segnali fisiologici grezzi dal prototipo di board sensoristica indossato dal paziente (ECG, IMU a 6 assi, temperatura), trasmessi via BLE al dongle nRF52840 collegato al PC di acquisizione tramite USB. L'applicazione dispone di un'interfaccia grafica nativa (Kivy) che visualizza in tempo reale i tracciati ECG, accelerometro, giroscopio e temperatura. L'integrazione con SmartCare avviene in modo non invasivo, senza richiedere modifiche al codebase esistente di IIT né alterare l'interfaccia grafica originale: il layer di comunicazione MQTT (`mqtt_bridge.py`) si aggancia direttamente al flusso di dati già acquisito dall'applicazione, pubblicandolo sul broker in parallelo alla normale visualizzazione a schermo.

5.1.2 Brokering ed Elaborazione

Al centro dell'architettura risiede il broker **Mosquitto**, che funge da hub asincrono per tutti i messaggi di telemetria. Il broker riceve un messaggio al secondo da ogni paziente su topic `smartcare/dati`, contenente una finestra di ECG (circa 10 intervalli R-R), una lettura di temperatura, e una finestra di 2 secondi di dati IMU (accelerometro + giroscopio a 6 assi). Questo flusso è cifrato su TLS (porta 8883).

Il processo `mqtt_subscriber.py` si sottoscrive a `smartcare/dati` e classifica ogni messaggio in tempo reale, tramite tre classificatori indipendenti (Strategy Pattern):

- **ECGClassifier**: Random Forest addestrato su `chfdb`, fornisce una probabilità di anomalia per la finestra cardiaca
- **PosturaClassifier**: Random Forest addestrato su `MHEALTH`, classifica l'attività motoria in 8 livelli (da fermo a corsa)
- **TemperaturaClassifier**: soglie cliniche deterministiche (ipotermia, normale, febbre, febbre alta)

Le letture classificate come anomale (score ECG superiore alla soglia 0,5) vengono:

1. salvate su MongoDB come documenti **Annotation**;
2. raggruppate in episodi clinici, unendo letture anomale consecutive con gap temporale inferiore a 10 secondi;
3. pubblicate sul topic `smartcare/allarmi` per notifiche real-time al medico e al paziente.

5.1.3 API

In parallelo al flusso di classificazione, il processo `fastapi_server.py` espone una REST API HTTPS (porta 8443) per tutte le operazioni sincrone: creazione e gestione profili, autenticazione, recupero dello storico, e validazione clinica degli episodi.

L'autenticazione distingue due ruoli:

- **Medico:** login tramite email e password, riceve un JWT (scadenza 8 ore) da includere in ogni richiesta successiva;
- **Paziente:** accesso tramite il codice univoco a 8 caratteri generato dal medico, controllato dal server al momento del login e mantenuto localmente dall'applicazione per la durata della sessione, senza un meccanismo di token equivalente a quello del medico.

5.1.4 Persistenza

I dati del sistema risiedono su due database distinti:

- **MongoDB:** tutte le annotazioni, episodi, e serie temporali di segnali anomali — struttura dinamica e adatta a query analitiche.
- **MySQL:** profili anagrafici di medici e pazienti, relazioni di cura, codici di accesso — dati strutturati e transazionali.

5.1.5 Validazione e Retraining

Il medico valida gli episodi anomali tramite la dashboard, classificandoli come *vero positivo* (anomalia reale) o *falso allarme*. Queste validazioni vengono accumulate su MongoDB. Ogni notte, il processo `retrain_scheduler.py` si attiva e:

1. Recupera le validazioni cliniche accumulate e, se in numero sufficiente, le combina con il dataset `chfdb` originario (mantenuto in una cache locale per evitare di riscaricarlo da PhysioNet ad ogni ciclo)
2. Ri-addestra il modello ECG sull'insieme combinato, valutandone le prestazioni su uno split di controllo prima del salvataggio
3. Sostituisce il modello in produzione tramite un meccanismo di *hot-reload* basato sul timestamp del file, senza interruzione del servizio di `mqtt_subscriber.py`

Questo ciclo chiude il loop tra intelligenza artificiale e giudizio clinico: il dataset di base garantisce che il modello non perda la conoscenza statistica appresa dai casi

clinici originari, mentre le validazioni correggono e specializzano progressivamente il comportamento del classificatore sui pazienti realmente monitorati.

5.1.6 Comunicazione Real-Time

La dashboard medico riceve le notifiche di allarme tramite due canali complementari:

1. **WebSocket (WSS) sul broker:** il topic `smartcare/allarmi` è esposto anche come listener WebSocket sulla porta 9002, consentendo alla dashboard di ricevere gli allarmi *istantaneamente*, senza necessità di polling;
2. **REST polling:** ogni 8 secondi, la dashboard interroga la API per aggiornare la lista completa degli episodi in attesa, garantendo coerenza anche in caso di disconnessioni MQTT transitorie.

La combinazione dei due meccanismi — MQTT per gli eventi istantanei, REST per la persistenza e il recupero dello storico — garantisce sia reattività sia affidabilità, senza dipendere esclusivamente da un singolo canale di comunicazione.

5.2 Pipeline di Acquisizione e Classificazione

Questa sezione descrive il percorso di un singolo messaggio di telemetria, dalla sua pubblicazione da parte dell'applicazione di acquisizione fino al salvataggio dell'annotazione risultante, illustrando come i tre classificatori vengono orchestrati in sequenza su ciascuna lettura.

5.2.1 Pubblicazione del Messaggio

Il layer di integrazione `mqtt_bridge.py` dell'app *BioDataAcq* pubblica, una volta al secondo e solo se l'acquisizione è attiva e il paziente ha effettuato l'accesso, un messaggio JSON sul topic `smartcare/dati`. Il payload contiene tre componenti eterogenee, raccolte nell'intervallo trascorso dall'ultima pubblicazione:

- una finestra di campioni ECG grezzi (circa 250 campioni, corrispondenti a un secondo di acquisizione a 250 Hz);
- una finestra di campioni IMU (accelerometro e giroscopio su 6 assi, circa 104 campioni a 104 Hz), convertiti dai conteggi raw del dongle nelle unità fisiche coerenti con il training (m/s^2 per l'accelerazione, gradi/s per la velocità angolare);
- l'ultima lettura di temperatura corporea disponibile.

5.2.2 Orchestrazione della Classificazione

Il processo `mqtt_subscriber.py`, sottoscritto al topic, riceve il messaggio e lo inoltra a un servizio di orchestrazione (`AnnotationService`), responsabile di coordinare i tre classificatori su ciascuna lettura:

1. **Segnale ECG.** Se il payload non contiene già gli intervalli R-R, questi vengono derivati dal segnale grezzo tramite individuazione dei picchi locali; la finestra risultante viene passata all'`ECGClassifier`, che restituisce l'etichetta (normale/anomalo) e il relativo score.
2. **Segnale di movimento.** L'intera finestra IMU ricevuta viene accodata al buffer interno del `PosturaClassifier`, mantenuto separatamente per ciascun paziente: la classificazione avviene solo quando il buffer raggiunge la dimensione richiesta dal modello (finestre di 2 secondi con sovrapposizione del 50%), così da restare coerente con le condizioni di addestramento anche a fronte di un tasso di arrivo dei messaggi diverso dalla finestra del classificatore.
3. **Temperatura corporea.** Il valore ricevuto viene confrontato direttamente contro le soglie cliniche del `TemperaturaClassifier`.

Gli esiti dei tre classificatori vengono infine raccolti in un'unica annotazione e persistiti su MongoDB.

5.2.3 Gestione della Finestra ECG Estesa

Quando la lettura corrente risulta anomala, il sistema avvia una procedura asincrona per costruire una traccia ECG estesa (circa 30 secondi, centrata sul picco anomalo) da mostrare al medico in fase di validazione. Poiché al momento della rilevazione sono disponibili solo i campioni precedenti l'anomalia, un componente dedicato (`ECGBufferManager`) mantiene una finestra scorrevole dei campioni ricevuti per ciascun paziente e completa la traccia non appena arrivano anche i campioni successivi all'evento, aggiornando l'annotazione già salvata.

5.2.4 Notifica e Raggruppamento in Episodi

Se la lettura corrente è anomala, un allarme viene pubblicato immediatamente sul topic `smartcare/allarmi`, ricevuto in tempo reale sia dalla dashboard medico sia dall'applicazione paziente. In fase di interrogazione, letture anomale consecutive dello stesso paziente con un intervallo inferiore a 10 secondi vengono raggruppate in un unico episodio clinico.

5.3 Moduli di Classificazione

I tre classificatori del sistema condividono un'interfaccia comune, secondo il pattern *Strategy*: la classe astratta `BaseClassifier` definisce un unico metodo `predict(data)`, che riceve un dizionario di dati grezzi e restituisce un'etichetta testuale con il relativo score di confidenza. Questa uniformità consente al resto del sistema di orchestrare i tre modelli in modo intercambiabile, senza conoscerne i dettagli implementativi, e rende possibile sostituire o affiancare in futuro un nuovo classificatore senza modificare il codice esistente.

5.3.1 ECGClassifier

Il classificatore ECG utilizza un modello Random Forest, addestrato sul dataset *chfdb* di PhysioNet, contenente registrazioni reali di pazienti con insufficienza cardiaca congestizia annotate a livello di singolo battito. In fase di addestramento, gli intervalli R-R vengono raggruppati in finestre di 10 battiti consecutivi, da cui si estraggono cinque feature statistiche (media, deviazione standard, minimo, massimo e range); una finestra è etichettata come anomala se contiene almeno un battito non normale. Il modello è addestrato con bilanciamento delle classi, per compensare la naturale prevalenza di battiti normali nel dataset.

A runtime, il classificatore riceve gli ultimi 10 intervalli R-R disponibili per il paziente ed effettua la stessa estrazione di feature utilizzata in addestramento; l'esito è un'etichetta binaria (normale/anomalo) determinata da una soglia sulla probabilità restituita dal modello, insieme al relativo score di confidenza.

Un aspetto rilevante del classificatore ECG è il meccanismo di *hot-reload*: poiché il ri-addestramento periodico avviene in un processo separato da quello di classificazione, ogni istanza del classificatore verifica, prima di ciascuna predizione, se il file del modello su disco è stato modificato rispetto all'ultima versione caricata in memoria, ricaricandolo in tal caso. Questo evita la necessità di un meccanismo di comunicazione esplicito tra i processi e permette di aggiornare il modello in produzione senza interrompere il servizio.

5.3.2 PosturaClassifier

Il classificatore di postura utilizza anch'esso un modello Random Forest, addestrato sul dataset MHEALTH, da cui vengono selezionate le sole registrazioni del sensore posizionato su braccio/polso — l'unico punto anatomico del dataset che espone contemporaneamente accelerometro e giroscopio, coerentemente con il sensore IMU del dispositivo reale. Il segnale continuo viene segmentato in finestre temporali, da ciascu-

na delle quali si estraggono venticinque feature statistiche (media, deviazione standard, minimo e massimo per ciascuno dei sei assi, più un'unica misura aggregata di energia del movimento su tutti gli assi). Il numero di classi di attività motoria è stato inoltre ridotto rispetto alle dodici originarie del dataset, mantenendo le otto ritenute rilevanti per il contesto clinico (dal riposo ad attività fisiche intense).

A runtime, il classificatore opera su finestre di due secondi con un avanzamento di un secondo tra una classificazione e la successiva (si veda la spiegazione dettagliata in nota¹); poiché ciascun messaggio MQTT trasporta solo una porzione di finestra, il classificatore mantiene uno stato interno tra un messaggio e l'altro, isolato per ciascun paziente, cosicché i campioni provenienti da pazienti diversi non vengano mai combinati nella stessa finestra pur condividendo la stessa istanza di classificatore.

5.3.3 TemperaturaClassifier

A differenza dei due classificatori precedenti, la componente relativa alla temperatura corporea non utilizza un modello statistico, bensì un insieme di soglie cliniche deterministiche, che ripartiscono il valore ricevuto in quattro categorie (ipotermia, normale, febbre, febbre alta). La scelta di un approccio a regole, anziché di un modello supervisionato, riflette la natura del segnale: la temperatura corporea possiede riferimenti clinici consolidati e universalmente accettati, per cui un modello addestrato su dati non offrirebbe un vantaggio significativo rispetto a soglie note a priori, mentre garantisce piena interpretabilità e assenza di necessità di addestramento o validazione statistica.

5.4 Persistenza dei Dati

Il sistema adotta due database con ruoli distinti, secondo il *Repository Pattern*: l'accesso ai dati non avviene mai direttamente dal codice applicativo, ma è incapsulato in classi dedicate (`AnnotationRepository` per MongoDB, `UserRepository` per MySQL), che espongono operazioni di alto livello (ad esempio *trova le anomalie non validate di un paziente*) nascondendo il dettaglio delle query sottostanti. Questo disaccoppiamento consente, in linea di principio, di sostituire il database sottostante senza modificare la logica applicativa che lo utilizza.

¹Il classificatore utilizza una finestra scorrevole con sovrapposizione del 50%: ogni nuova predizione riutilizza la metà più recente dei campioni già osservati e vi aggiunge un secondo di dati nuovi, garantendo una predizione al secondo senza frammentare un singolo evento motorio tra due finestre distinte.

5.4.1 MongoDB: Annotazioni

MongoDB ospita l'intero flusso di annotazioni generato dalla pipeline di classificazione (Sezione 5.2): ogni lettura — normale o anomala — viene salvata come documento contenente il timestamp, l'esito e lo score di ciascuno dei tre classificatori, ed eventualmente l'esito della validazione clinica una volta effettuata dal medico.

La natura documentale di MongoDB si adatta bene a questo caso d'uso per due motivi. Anzitutto, non tutte le annotazioni hanno la stessa struttura: solo le letture classificate come anomalie ECG vengono arricchite, in un secondo momento e in modo asincrono, con la finestra estesa del tracciato, mentre le letture normali non possiedono affatto questo campo. In un database relazionale, questa differenza andrebbe gestita con una tabella separata oppure con colonne che restano vuote nella maggior parte dei record; MongoDB, non imponendo uno schema rigido, rappresenta questa variabilità in modo naturale, permettendo a un documento di includere campi aggiuntivi rispetto a un altro dello stesso tipo. In secondo luogo, le interrogazioni più frequenti (storico di un paziente, anomalie non validate, raggruppamento in episodi) accedono quasi sempre al documento nella sua interezza, senza richiedere le *join* tipiche di uno schema relazionale normalizzato.

Il raggruppamento delle letture anomale in episodi clinici, già descritto precedentemente, avviene interamente a livello applicativo all'interno del repository: le letture vengono recuperate ordinate cronologicamente e aggregate scorrendole in sequenza, anziché tramite un'aggregazione nativa del database. Questa scelta privilegia la leggibilità della logica di raggruppamento rispetto alle prestazioni, giustificata dal volume contenuto di documenti in attesa di validazione rispetto allo storico complessivo.

5.4.2 MySQL: Profili Anagrafici

MySQL ospita invece i dati anagrafici di medici e pazienti e la relazione di cura che li lega. A differenza delle annotazioni, questi dati sono strutturati, di volume contenuto e soggetti a vincoli di integrità ben definiti: ogni paziente appartiene a un solo medico, e il codice di accesso generato per ciascun paziente deve essere univoco nell'intero sistema. La natura relazionale di MySQL consente di imporre questi vincoli direttamente a livello di schema — tramite chiave esterna e vincolo di unicità — anziché delegarli interamente alla logica applicativa, riducendo il rischio di incoerenze in caso di accessi concorrenti.

5.5 Ciclo di Validazione Clinica

Ogni episodio anomalo generato dalla pipeline di classificazione (Sezione 5.2) rimane in attesa fino a quando un medico non lo esamina dalla dashboard. Questa sezione descrive l'implementazione del ciclo di validazione, dal recupero degli episodi non validati fino alla propagazione dell'esito al paziente.

5.5.1 Recupero degli Episodi

L'endpoint `GET /anomalie/episodi`, protetto da autenticazione JWT, invoca il metodo `get_episodi_anomalia_non_validati()` del servizio `AnnotationService`, che a sua volta delega al repository il raggruppamento in episodi descritto nella Sezione 5.2 filtrando esclusivamente i documenti con campo `esito_medico` non valorizzato. Per ciascun episodio, il backend arricchisce la risposta con nome e cognome del paziente recuperati da MySQL (funzione `_arricchisci_con_dati_paziente`), cosicché la dashboard non debba effettuare una seconda richiesta per ottenere questi dati. La dashboard interroga questo endpoint sia al montaggio del componente sia a intervalli di 8 secondi (`POLLING_EPISODI_MS`), aggiornando lo stato React che alimenta la coda visualizzata.

Se l'episodio riguarda un'anomalia ECG, il componente `ValidationModal` seleziona automaticamente, tra i documenti del cluster, quello con lo score più alto per mostrarne la traccia estesa. Qualora il campo `ecg_window_pronta` risulti ancora falso — perché il buffer descritto nella Sezione 5.2 non ha ancora accumulato i campioni successivi al picco — l'interfaccia espone un pulsante che richiama l'endpoint `GET /annotazioni/{id}` per un nuovo tentativo, senza impedire nel frattempo la validazione dell'episodio sulla base dei soli dati già disponibili.

5.5.2 Applicazione della Validazione

La conferma da parte del medico invoca l'endpoint `PATCH /anomalie/episodi/valida`, che riceve l'elenco degli identificativi delle letture appartenenti all'episodio, l'esito (`vero_positivo` o `falso_allarme`) e una nota testuale opzionale.

Il metodo `valida_episodio()` del servizio `AnnotationService` delega al repository l'aggiornamento (`update_esito_medico_multiplo`), che scrive lo stesso esito e la stessa nota su ciascun documento del cluster in una singola operazione `update_many` su MongoDB: il medico esprime così un'unica decisione per l'intero episodio, mentre il dato resta granulare per singola lettura — condizione necessaria affinché il retraining, descritto nella sezione successiva, possa operare sulle singole letture validate anziché sull'aggregato.

Subito dopo la scrittura, l'endpoint invoca il metodo `notifica_validazione()` del servizio `NotificationService`, che pubblica un messaggio sul topic `smartcare/validazioni`, distinto da `smartcare/allarmi` poiché veicola una struttura del payload differente (esito, nota, numero di letture coinvolte, anziché gli score dei classificatori). L'applicazione paziente, sottoscritta a questo topic, riceve così l'esito in tempo reale, senza dover effettuare polling sullo storico per accorgersene.

5.5.3 Storico e Rivalutazione

Un episodio già validato non viene rimosso dallo storico: l'endpoint pubblico `GET /pazienti/by-codice/{codice}/episodi`, utilizzato sia dalla sezione Storico della dashboard sia dall'applicazione paziente, restituisce — a differenza di `GET /anomalie/episodi` — sia gli episodi in attesa sia quelli già validati, includendone nota ed esito. Il medico può quindi riaprire dallo storico un episodio già valutato e richiamare nuovamente l'endpoint di validazione con un esito diverso.

5.6 Retraining Periodico

Le validazioni cliniche accumulate alimentano un processo di ri-addestramento periodico del classificatore ECG, che costituisce il punto di chiusura del ciclo tra intelligenza artificiale e giudizio clinico. Questa sezione ne descrive l'implementazione: la schedulazione, l'estrazione delle feature dai dati validati, e il meccanismo con cui il nuovo modello viene messo in produzione senza interrompere il servizio.

5.6.1 Schedulazione

Il ri-addestramento è gestito da un processo standalone (`retrain_scheduler.py`), separato sia da `mqtt_subscriber.py` sia da `fastapi_server.py`, e schedulato tramite **APScheduler** con un job notturno a orario configurabile. Il processo utilizza uno scheduler in background anziché bloccante.

Il job può inoltre essere eseguito immediatamente in modalità manuale (flag `-now`), utile in fase di test per non dover attendere l'orario notturno pianificato. Un margine di tolleranza (`misfire_grace_time`) consente inoltre al job di essere eseguito comunque se il sistema era spento all'orario previsto, purché il ritardo non superi un'ora.

5.6.2 Estrazione del Dataset ed Addestramento

Ad ogni attivazione, `RetrainService` recupera dal repository tutte le annotazioni con campo `esito_medico` valorizzato — le letture già validate dal medico, indipendentemente dall'episodio di appartenenza — e ne estrae, per ciascuna, le stesse cinque

feature statistiche sugli intervalli R-R utilizzate in fase di addestramento iniziale (Sezione 5.3). Le etichette di training vengono derivate direttamente dall'esito medico: una lettura validata come *vero positivo* diventa un'osservazione positiva, una validata come *falso allarme* diventa un'osservazione negativa — indipendentemente dall'etichetta originariamente assegnata dal classificatore, che viene così corretta dal giudizio clinico.

Queste feature vengono quindi **unite** (non sostituite, come avveniva in una prima versione del meccanismo) a quelle del dataset `chfdb` impiegato nell'addestramento iniziale, mantenute in una cache locale su disco per evitare di ricontattare PhysioNet e ricalcolare le finestre di intervalli R-R ad ogni ciclo notturno; la cache viene invalidata automaticamente qualora la logica di estrazione delle feature venga in futuro modificata. Il Random Forest viene quindi ri-addestrato sull'intero insieme combinato, garantendo che la conoscenza statistica di base non venga mai persa e che le validazioni mediche si limitino ad arricchirla con segnale clinico specifico ai pazienti realmente monitorati.

Il ri-addestramento viene comunque eseguito solo se il numero di nuove validazioni disponibili supera una soglia minima configurabile (dieci osservazioni), usata unicamente come condizione di attivazione — non come dimensione del dataset di addestramento — volta a evitare cicli notturni superflui quando non è ancora giunto un volume di segnale clinico sufficiente a giustificarli. Prima di procedere al salvataggio, il sistema valuta le prestazioni del modello su uno split di controllo (80/20) del dataset combinato, producendo un report di classificazione utile a monitorare nel tempo l'effetto delle nuove validazioni sull'accuratezza complessiva; il modello effettivamente distribuito è tuttavia addestrato sull'insieme completo, riservando lo split esclusivamente alla fase di valutazione.

5.6.3 Sostituzione Atomica e Hot-Reload

Poiché il processo di retraining e i processi di classificazione (`mqtts_subscriber.py`, `fastapi_server.py`) sono indipendenti e non condividono memoria, la propagazione del nuovo modello non avviene tramite comunicazione diretta tra processi, ma attraverso il filesystem. Il nuovo modello viene scritto su un file temporaneo nella stessa cartella di destinazione e poi spostato sopra il file `.pkl` definitivo tramite `os.replace()`, operazione atomica sullo stesso filesystem: questo garantisce che un processo che legga il file non lo trovi mai a metà scrittura, evitando un caricamento corrotto o parziale durante il ricaricamento a caldo.

Ogni istanza di `ECGClassifier`, prima di ciascuna predizione, confronta il timestamp di ultima modifica (`mtime`) del file del modello con quello registrato al momento del-

l'ultimo caricamento; se risulta più recente, il modello viene ricaricato da disco. Il costo di questo controllo si riduce a una singola chiamata di sistema (`os.stat()`) per predizione, trascurabile rispetto al tempo di inferenza stesso. Se il file risultasse temporaneamente non accessibile — ad esempio durante la finestra di scrittura del nuovo modello — il classificatore continua a operare con la versione già in memoria, rimandando il tentativo di ricarica alla predizione successiva, senza mai interrompere il servizio di classificazione in corso.

5.7 REST API e Autenticazione

Il backend espone una REST API tramite FastAPI, in ascolto su HTTPS (porta 8443), che copre tutte le operazioni sincrone del sistema: autenticazione, gestione dei profili, recupero dello storico e validazione clinica. Questa sezione descrive i meccanismi di autenticazione adottati e gli endpoint principali, organizzati per area funzionale.

5.7.1 Autenticazione

Il sistema distingue due meccanismi di accesso, coerentemente con la natura differente dei due ruoli.

Il medico si autentica tramite l'endpoint `POST /auth/login`, che accetta email e password (verificata contro l'hash memorizzato con **bcrypt**) e restituisce, in caso di successo, un **JWT** con scadenza di otto ore. Il token viene incluso dal client in ogni richiesta successiva tramite l'header `Authorization`, e la sua validità è verificata da una dipendenza FastAPI (`get_medico_corrente`) condivisa da tutti gli endpoint riservati al medico, che ne decodifica il contenuto e risolve il profilo corrispondente su MySQL.

Il paziente, non disponendo di credenziali tradizionali, accede fornendo il codice univoco a otto caratteri generato dal medico al momento della creazione del profilo. Il codice viene controllato dal server sull'endpoint pubblico `GET /pazienti/by-codice/{codice}`, privo di autenticazione JWT poiché il codice stesso — non facilmente indovinabile e generato in modo casuale — costituisce già una forma minima di credenziale; l'esito viene mantenuto localmente dall'applicazione paziente per la durata della sessione.

5.7.2 Endpoint Principali

Gli endpoint riservati al medico richiedono il token JWT e operano principalmente sui dati identificati dall'id numerico del paziente in MySQL:

- `POST /pazienti`: crea un nuovo profilo paziente, generando automaticamente un codice di accesso univoco;
- `GET /pazienti`: restituisce l'elenco dei pazienti in carico al medico autenticato;
- `GET /anomalie/episodi`: restituisce gli episodi non validati, raggruppati clinicamente (Sezione 5.5);
- `GET /pazienti/{id}/storico`: restituisce lo storico completo delle letture di un paziente;
- `PATCH /anomalie/episodi/valida`: applica una validazione a un intero episodio.

Un secondo gruppo di endpoint, pubblici e non protetti da JWT, è pensato per l'applicazione paziente e identifica il paziente tramite il codice di accesso anziché tramite l'id numerico:

- `GET /pazienti/by-codice/{codice}`: valida il codice e restituisce i dati anagrafici essenziali;
- `GET /pazienti/by-codice/{codice}/episodi`: restituisce gli episodi — sia in attesa sia già validati — relativi al paziente, con lo stesso raggruppamento clinico usato dalla dashboard.

5.8 Dashboard Medico — React

La dashboard medico consuma esclusivamente le API REST e il canale MQTT esposti dal backend descritto nei paragrafi precedenti, senza alcuna logica di dominio propria: ogni classificazione, raggruppamento in episodi e validazione avviene lato server, mentre il client si limita a orchestrare stato locale, presentazione e due canali di aggiornamento paralleli — REST polling e WebSocket — che si compensano a vicenda.

5.8.1 Doppio canale di aggiornamento: polling REST e WebSocket MQTT

Una scelta architetturale che ricorre in tutta la dashboard è l'uso combinato di due meccanismi di aggiornamento con scopi complementari, orchestrati in `Dashboard.jsx`:

- **Polling REST**, per gli episodi non validati e per la lista pazienti. Garantisce che lo stato mostrato converga sempre a quello reale del backend, anche in assenza di eventi MQTT (es. dopo una disconnessione temporanea del broker) o all'apertura iniziale della pagina.

- **WebSocket MQTT** (hook² `useMqtt`), sottoscritto al topic `smartcare/allarmi`, per la notifica *istantanea* di una nuova anomalia — toast, notifica desktop e beep sonoro entro 1-2 secondi dall’evento, molto più reattivo degli 8 secondi del polling.

Il WebSocket non sostituisce dunque il polling, ma lo integra: alla ricezione di un allarme MQTT, se la sezione attiva è `anomalie` o `panoramica`, viene forzato un refresh anticipato (`setTimeout(caricaEpisodi, 800)`), scelto con un breve ritardo per dare al backend il tempo di completare la scrittura MongoDB dell’annotazione prima della lettura REST successiva.

5.8.2 Connessione WebSocket — `useMqtt`

L’hook `useMqtt` incapsula la libreria `mqtt.js`, connettendosi al broker Mosquitto sul listener WebSocket con TLS (porta 9002, protocollo `wss://`).

- Il `clientId` è generato con un suffisso casuale ad ogni mount, evitando collisioni se il medico apre la dashboard in più schede del browser contemporaneamente — un `clientId` duplicato causerebbe la disconnessione forzata della sessione precedente da parte del broker, come previsto dallo standard MQTT.
- Lo stato di connessione esposto (`connecting` / `connected` / `disconnected`) alimenta l’indicatore visivo in `Topbar.jsx` (pallino colorato + etichetta), dando al medico un riscontro immediato sulla salute del canale real-time senza dover interpretare messaggi di errore tecnici.
- Il `reconnectPeriod: 3000` delega alla libreria stessa la riconnessione automatica in caso di caduta del broker, senza necessità di logica applicativa aggiuntiva: allo stop e riavvio di Mosquitto, il pallino transita autonomamente da `disconnected` a `connecting` a `connected`, senza ricaricare la pagina.
- Il cleanup dell’effetto (`client.end(true)`) è essenziale sotto `React.StrictMode`, che monta/smonta gli effetti due volte in sviluppo: senza una disconnessione esplicita alla distruzione, ogni doppio mount lascerebbe una connessione MQTT orfana attiva in background.

5.8.3 Debounce degli allarmi

Un allarme MQTT viene pubblicato dal backend per *ogni singola lettura anomala*, non per episodio: un episodio di fibrillazione atriale di 30 secondi genera quindi fino a 30

²In React, funzione che permette a un componente di usare stato o effetti collaterali (es. sottoscrizioni, connessioni di rete).

messaggi MQTT consecutivi. Senza un meccanismo di deduplica lato client, il medico riceverebbe 30 notifiche desktop e 30 beep per lo stesso evento clinico.

La costante `DEBOUNCE_ALLARME_MS = 15000` in `Dashboard.jsx`, applicata tramite un `useRef` (`ultimoAllarmePerPaziente`), sopprime le notifiche ripetute per lo stesso paziente entro quella finestra temporale. Il valore (15s) è stato scelto per eccedere leggermente il `GAP_MASSIMO_EPISODIO_SECONDI = 10` usato lato server per il clustering in episodi: questo garantisce che, nel caso più comune di letture consecutive con gap regolare (1 al secondo), il client tratti l'intero episodio come un singolo evento notificabile, coerentemente con quanto la vista **Anomalie** mostrerà, già raggruppato lato server. Il toast in-app (a differenza della notifica desktop e del beep) non è invece soggetto a debounce: resta visibile per ogni singolo messaggio ricevuto, offrendo un log visivo più granulare per chi tiene la scheda in primo piano.

5.8.4 Gestione della sessione e scadenza token — `apiFetch`

Tutte le chiamate REST autenticate passano per `apiFetch` (`api/client.js`), un sottile wrapper attorno a `fetch` che centralizza due responsabilità:

1. L'header `Authorization: Bearer <token>`, letto dal `AuthContext` e allegato automaticamente ad ogni richiesta, evitando che ciascun componente debba occuparsene individualmente.
2. L'intercettazione dello stato 401: se il JWT è scaduto o non più valido, `apiFetch` invoca il callback `onUnauthorized` (tipicamente `logout()` da `AuthContext`) e restituisce `null` al chiamante, che deve quindi limitarsi a un controllo `if (!res || !res.ok) return` senza duplicare la logica di redirect al login.

Gli endpoint pubblici consumati dall'app paziente (`/pazienti/by-codice/...`) passano invece per `publicFetch`, priva di header di autenticazione — coerente con la scelta architetturale già discussa precedentemente di non richiedere JWT per quel flusso.

5.8.5 Modal di validazione unificato

Una particolarità dell'implementazione React rispetto a una dashboard con pagine distinte è l'uso di un *singolo* componente, `ValidationModal`, per due scenari concettualmente diversi mostrati con la prop `modalita`:

- `'valida'`: aperto dalla sezione **Anomalie** (o dal riepilogo in **Panoramica**) su un episodio non ancora validato — mostra i due pulsanti di esito senza banner di stato pregresso.

- `'storico'`: aperto dalla sezione **Storico** su un episodio già validato o meno — mostra in aggiunta un **BannerEsito** con l'esito corrente e la data di validazione, e consente di modificarlo tramite lo stesso flusso di conferma.

Entrambe le modalità condividono la medesima funzione di conferma in `Dashboard.jsx`, che distingue il comportamento post-salvataggio in base a `modalita`: solo se `modalita === 'storico'` viene incrementato un `storicoRefreshSignal`, un contatore osservato dal componente **Storico** per auto-ricaricare la lista dopo una modifica, senza che l'utente debba ripremere manualmente "Carica".

5.8.6 Traccia ECG estesa — costruzione asincrona lato client

Il componente `EcgChart` riflette direttamente, sul lato presentazione, la natura asincrona della finestra ECG estesa descritta in Sezione 5.2: se `ecg_window_pronta` è ancora `false` al momento dell'apertura del modal (il backend non ha ancora ricevuto abbastanza campioni successivi al picco anomalo), viene mostrato un messaggio di attesa con un pulsante **Riprova**, che richiama `GET /annotazioni/{annotation_id}` per verificare se la finestra è nel frattempo diventata disponibile — senza dover chiudere e riaprire il modal.

Per gli episodi che raggruppano più letture, la funzione `scegliDocumentoPerGrafico` (`utils/format.js`) seleziona per il grafico non la prima né l'ultima lettura del cluster, ma quella con `ecg_score` più alto — il momento clinicamente più significativo dell'episodio, coerente con il criterio già usato lato server.

5.8.7 Notifiche desktop e feedback sonoro

L'hook `useNotifiche` incapsula la Web Notification API del browser, con una distinzione voluta tra *permesso di sistema* (irrevocabile via JavaScript una volta negato) e *abilitazione applicativa* (`abilitate`, un semplice stato booleano locale che sopprime le chiamate a `new Notification()` quando l'utente disattiva il bottone in **Topbar** senza dover revocare il permesso del browser stesso) — lo stesso pattern adottato da client di posta e messaggistica affermati. Il beep sonoro (`suonaAllarme`, tre toni sintetizzati via Web Audio API) è invece sempre attivo alla ricezione di un allarme MQTT, indipendentemente dallo stato delle notifiche desktop, fornendo un canale di allerta minimo anche quando il permesso di sistema è stato negato o non è supportato dal browser.

5.9 Integrazione App Paziente

L'app paziente **IIT BioDataAcq** è di proprietà dell'Istituto Italiano di Tecnologia; il presente lavoro descrive nel dettaglio esclusivamente il *layer di integrazione* realizzato per questo progetto: un insieme di quattro moduli Python (`mqtt_bridge.py`, `patient_login.py`, `patient_session.py`, `patient_anomalies.py`) che si agganciano al core esistente dell'app senza modificarlo, secondo un principio di **non invasività** illustrato più avanti.

5.9.1 Sessione paziente — `patient_session.py`

Il modulo più semplice dell'integrazione è anche quello architetturealmente più centrale: un registro in memoria a livello di modulo (pattern *Singleton implicito*) che conserva il codice di accesso del paziente attualmente loggato. Questo modulo funge da *fonte di verità condivisa* fra gli altri tre: `mqtt_bridge.py` lo consulta ad ogni tick per decidere se pubblicare, `patient_anomalies.py` lo consulta per filtrare gli eventi MQTT in arrivo e per interrogare l'endpoint pubblico corretto, e `patient_login.py` lo popola al termine dell'autenticazione. Senza questo registro, ciascuno dei tre moduli dovrebbe negoziare autonomamente lo stato di login, con rischio di disallineamento.

5.9.2 Login paziente — `patient_login.py`

Il login lato paziente non richiede una vera autenticazione con password: l'utente digita il codice di accesso a 8 caratteri fornito dal medico in un popup dedicato (`PatientLoginPopup`), e il modulo lo verifica interrogando il backend tramite l'endpoint pubblico `GET /pazienti/by-codice/codice`:

```
res = requests.get(f"{API_BASE}/pazienti/by-codice/{codice}", timeout=5,
verify=CA_CERT_PATH)
```

Da segnalare l'uso esplicito di `verify=CA_CERT_PATH`: poiché il certificato TLS di `FastAPI` è firmato dalla CA locale `mkcert` e non da un'autorità pubblica riconosciuta, senza questo parametro esplicito la chiamata fallirebbe con un errore di verifica del certificato. Se il codice risulta valido (200 OK), la risposta JSON — contenente nome, cognome e codice del paziente — viene passata a `patient_session.set_paziente()`, e un popup di benvenuto (`mostra_popup_benvenuto`) conferma l'accesso mostrando il nome del paziente per 6 secondi, con possibilità di chiusura anticipata.

5.9.3 Bridge MQTT — `mqtt_bridge.py`

`MQTTBridge` è il componente che collega il core di acquisizione esistente dell'app (i `PlotWidget` che bufferizzano i campioni letti dal dongle) al topic `smartcare/dati`. La

sua caratteristica progettuale principale è la **non invasività**: non modifica né estende le classi esistenti dell'app (`PlotController`, `PlotWidget`), ma le legge periodicamente dall'esterno tramite un registry già presente (`registry.get_plot_controller()`, `registry.get_com_panel()`).

Un timer Kivy (`Clock.schedule_interval`, un secondo di periodo) esegue `_tick()`, che pubblica un messaggio solo se sono verificate *entrambe* le condizioni:

1. l'acquisizione è stata avviata dall'utente (`com_panel.started_acq`);
2. un paziente è loggato (`patient_session.get_codice()` non nullo).

Conversione delle unità fisiche. Il punto più delicato del bridge è la conversione dei conteggi raw a 16 bit del dongle nelle stesse unità fisiche usate in fase di training del `PosturaClassifier`: accelerazione in m/s^2 , velocità angolare in gradi/s, coerenti con la documentazione ufficiale di MHEALTH.

Finestra IMU completa. Analogamente a quanto fatto per l'ECG, la funzione `_leggi_imu_finestra()` restituisce l'intera finestra di campioni raccolti nell'ultimo secondo (tipicamente 104 a 104Hz), non solo l'ultimo valore.

5.9.4 Notifiche e storico anomalie — `patient_anomalies.py`

Il modulo più corposo dell'integrazione si divide in due parti indipendenti, entrambe centrate sull'esperienza del paziente rispetto agli eventi clinici che lo riguardano:

AnomaliesMonitor — badge sempre attivo. Un client MQTT separato dal bridge (quello pubblica, questo sottoscrive) rimane sempre connesso, indipendentemente dall'apertura di un popup, sottoscritto sia a `smartcare/allarmi` sia a `smartcare/validazioni`. Mantiene un contatore di eventi "non visti" che alimenta il badge sul bottone del menu superiore dell'app, filtrando i messaggi in arrivo per includere solo quelli riferiti al `paziente_id` attualmente loggato:

Per gli allarmi, viene applicato lo stesso principio di deduplica per episodio già visto lato dashboard React (Sezione 5.8): letture anomale consecutive entro un certo gap (10s, identico alla soglia di clustering server-side) contano come un solo evento, evitando che un episodio di 30 letture incrementi il badge di 30 unità. Le validazioni mediche, al contrario, non richiedono debounce: ogni messaggio su `smartcare/validazioni` rappresenta già un evento discreto (un medico ha appena validato un episodio) e viene sempre conteggiato, accompagnato da un beep sonoro e da un popup toast (`_mostra_toast_validazione`) che comunica l'esito.

Beep sintetizzato senza asset audio. Poiché Kivy non espone un equivalente della Web Audio API usata dalla dashboard React per generare il beep di allarme al volo (Sezione 5.8), il modulo ricrea lo stesso pattern sonoro (tre toni a 880Hz, 880Hz, 1100Hz con inviluppo esponenziale) campione per campione tramite la libreria standard `wave`, salvando il risultato in un file WAV temporaneo generato una sola volta e cachato in `tempfile.gettempdir()`. Questa scelta evita di dover distribuire un asset audio nel bundle dell'app, riproducendo comunque un'esperienza sonora coerente fra i due client.

AnomaliesPopup — storico on-demand. Aperto esplicitamente dall'utente, interroga l'endpoint pubblico `GET /pazienti/by-codice/{codice}/episodi` tramite una richiesta eseguita su un thread separato (`threading.Thread(daemon=True)`) per non bloccare il thread principale dell'interfaccia Kivy durante l'attesa della risposta di rete. A differenza del monitor, che mostra solo un contatore aggregato, il popup elenca ogni episodio con il proprio intervallo temporale, lo score ECG di picco e medio, e — se già presente — l'esito medico con eventuale nota clinica. L'apertura del popup azzerava contestualmente il contatore di `AnomaliesMonitor` (`reset_contatore()`), sul presupposto che l'utente abbia così "visto" tutti gli eventi accumulati fino a quel momento.

5.9.5 Considerazioni conclusive sull'integrazione

L'intero layer descritto in questa sezione condivide un principio progettuale comune: **aggancio non invasivo** a un'applicazione preesistente di cui non si controlla il codice sorgente. Nessuno dei quattro moduli modifica le classi originali dell'app IIT BioDataAcq; ciascuno vi si appoggia dall'esterno, tramite il registry già esposto dall'applicazione (`registry.py`) o tramite un semplice modulo di stato condiviso (`patient_session.py`). Questo pattern ha permesso di sviluppare e testare l'intera integrazione — inclusa la conversione delle unità fisiche IMU e la costruzione dei payload MQTT — senza necessità di intervenire sul codice proprietario IIT, riducendo al minimo l'accoppiamento fra i due repository e semplificando l'eventuale manutenzione futura da parte dei rispettivi proprietari del codice.

5.10 Considerazioni di Sicurezza

Questa sezione raccoglie trasversalmente le scelte di sicurezza e robustezza adottate nel sistema, alcune già introdotte nelle sezioni precedenti a proposito dei singoli componenti, ma qui discusse nel loro insieme: la cifratura del trasporto end-to-end, l'isolamento dei dati fra pazienti diversi condivisi dagli stessi processi, e la gestione degli errori nei punti del sistema più esposti a race condition o fallimenti esterni.

5.10.1 TLS end-to-end

Tutte le comunicazioni di rete del sistema sono cifrate, senza eccezioni: MQTT fra dispositivo e subscriber (8883), WebSocket fra broker e dashboard browser (9002), REST fra client e backend (8443, HTTPS). La cifratura si appoggia in ogni punto agli stessi certificati generati con **mkcert**, la cui CA locale viene installata nel trust store del sistema operativo di sviluppo, eliminando gli avvisi di sicurezza tipici dei certificati self-signed generati manualmente con OpenSSL.

La configurazione di Mosquitto (`mosquitto.conf`) espone deliberatamente tre listener distinti:

- **1883, in chiaro:** mantenuto esclusivamente per il debug locale rapido.
- **8883, con TLS:** usato dai client MQTT nativi (`paho-mqtt`), cioè il flusso dispositivo → backend.
- **9002, WebSocket con TLS:** usato esclusivamente dalla dashboard browser, che non può aprire una connessione MQTT nativa mancando l'accesso ai socket TCP grezzi dal contesto JavaScript.

Un'unica coppia di certificati (`server.crt/server.key`) copre tutti e tre i listener MQTT più il server FastAPI, centralizzando la gestione dei certificati invece di generarne uno diverso per servizio. È importante osservare che questa architettura implementa **TLS solo lato server**, non una mutua autenticazione TLS (mTLS): i client (app paziente, dashboard, subscriber) necessitano unicamente del certificato della CA (`ca.crt`) per verificare l'identità del server, ma non possiedono un proprio certificato client. La logica di configurazione TLS lato client è centralizzata in un unico modulo di utilità, `mqtt_tls.py`, riutilizzato da tutti i processi che aprono connessioni MQTT (`mqtt_subscriber.py`, `notification_service.py`, `simulate_stream.py`, `mqtt_bridge.py`, `patient_anomalies.py`), evitando di duplicare la configurazione SSL in ciascuno di essi.

5.10.2 Isolamento dei dati per paziente in processi condivisi

Diversi componenti del backend sono istanziati *una sola volta* e condivisi globalmente da `mqtt_subscriber.py` per tutti i pazienti connessi, per evitare l'overhead di ricaricare un modello di Machine Learning o riaprire una connessione ad ogni messaggio. Questa condivisione introduce però un rischio strutturale: se lo stato interno di un componente condiviso non viene tenuto esplicitamente separato per paziente, i dati di pazienti diversi possono mescolarsi nella stessa struttura dati, con conseguenze che vanno dalla semplice inefficienza fino a classificazioni cliniche errate.

Il sistema affronta questo rischio con un pattern ricorrente — un dizionario indicizzato per `paziente_id` al posto di una singola variabile di stato — applicato in almeno tre punti distinti:

- **PosturaClassifier**: il buffer della sliding window per l'accumulo dei campioni IMU, l'ultima etichetta predetta e l'ultimo score sono tenuti in tre dizionari (`_buffers`, `_ultima_label`, `_ultimo_score`) chiavizzati per `paziente_id`, protetti da un `threading.Lock` condiviso.
- **ECGBufferManager**: analogamente, il buffer scorrevole dei campioni ECG raw usato per costruire la finestra estesa attorno a un'anomalia è tenuto in due dizionari (`_buffers`, `_totali`) chiavizzati per paziente, protetti da un `Lock` dedicato.
- **AnnotationService**: la lista delle estrazioni di finestra ECG "in attesa" (anomalie salvate ma la cui finestra estesa non è ancora pronta) è anch'essa un dizionario per `paziente_id` (`_estrazioni_in_attesa`).

Questo pattern evita che il sistema scali in modo scorretto quando più pazienti sono monitorati contemporaneamente. Resta un'area di attenzione per il futuro l'assenza di un meccanismo di pulizia automatica di questi dizionari alla disconnessione di un paziente: lo stato resta in memoria per l'intera vita del processo, un compromesso accettabile con un numero limitato di pazienti ma da rivedere per deployment con una popolazione di pazienti più ampia e variabile nel tempo.

5.10.3 Gestione degli errori in scenari concorrenti e di rete instabile

Diversi punti del sistema sono stati progettati per tollerare esplicitamente condizioni di errore prevedibili, piuttosto che propagare un'eccezione che interromperebbe un processo di lunga durata:

Hot-reload del modello ECG. `ECGClassifier._controlla_reload()` confronta ad ogni predizione il `mtime` del file `.pkl` con quello dell'ultimo caricamento, ricaricando il modello se è cambiato — meccanismo che permette a `retrain_scheduler.py` di sostituire il modello in produzione senza richiedere il riavvio di `mqtt_subscriber.py` o `fastapi_server.py`. Sia il fallimento della `os.stat()` (file temporaneamente non disponibile) sia il fallimento del successivo `joblib.load()` (file potenzialmente a metà scrittura, in race con il salvataggio di `RetrainService`) vengono intercettati senza propagare l'eccezione: il processo continua a servire predizioni con il modello già in memoria, ritentando il reload alla chiamata successiva.

Salvataggio atomico del modello. `RetrainService._salva_modello_atomico()` scrive il nuovo modello su un file temporaneo nella stessa cartella di destinazione e lo rinomina sopra il file definitivo tramite `os.replace()`, operazione atomica sullo stesso filesystem. Questo garantisce che il meccanismo di hot-reload descritto sopra non trovi mai il file a metà scrittura: o legge la versione precedente, o legge quella completamente nuova, mai uno stato intermedio corrotto.

Fallback su dataset chfdb non disponibile. Se la cache locale delle feature chfdb non esiste ancora e PhysioNet non è raggiungibile al momento del retrain notturno, `RetrainService.ritrain()` intercetta l'eccezione e restituisce **False** invece di propagarla, saltando in modo sicuro il ciclo di quella notte piuttosto che addestrare un modello solo sulle validazioni mediche accumulate — scenario esplicitamente considerato indesiderabile, dato che comprometterebbe la base statistica fornita da chfdb.

Job di scheduling isolati. In `retrain_scheduler.py`, il job notturno è avvolto in un blocco `try/except` che intercetta qualunque eccezione e la logga senza interrompere lo scheduler stesso: un fallimento imprevisto in una singola esecuzione non deve compromettere le esecuzioni pianificate delle notti successive.

Scadenza e validità del token JWT. Sul fronte dell'autenticazione, un token scaduto, manomesso, o riconducibile a un medico non più esistente sul database produce sempre un **401 Unauthorized** gestito uniformemente da `get_medico_corrente`, mai un errore generico che esporrebbe dettagli implementativi al client.

Nel loro insieme, questi meccanismi riflettono un principio comune: nei punti del sistema più esposti a concorrenza (scrittura/lettura simultanea di un file di modello) o a dipendenze esterne inaffidabili (rete, servizi di terze parti come PhysioNet), la strategia adottata privilegia la **disponibilità del servizio** rispetto alla propagazione rigida dell'errore, un compromesso appropriato per un sistema di monitoraggio clinico continuo in cui un'interruzione del servizio di classificazione — anche temporanea — è più dannosa di una predizione temporaneamente basata su un modello leggermente meno aggiornato.

Capitolo 6

Validazione Funzionale e Prestazionale

Questo capitolo presenta i risultati della validazione del sistema SmartCare, condotta su due fronti complementari. Da un lato, la validazione **funzionale end-to-end**, effettuata utilizzando il dispositivo wearable reale — prototipo di board sensoristica IIT collegato tramite dongle nRF52840 — per verificare che l'intero flusso, dall'acquisizione del segnale fisiologico fino alla validazione clinica in dashboard, funzioni correttamente con dati realmente acquisiti dal sensore, e non solo con dati sintetici. Lo script `simulate_stream.py` resta uno strumento utile nelle fasi di sviluppo per testare la pipeline di classificazione in assenza del dispositivo fisico, ma non è stato impiegato per la validazione qui riportata.

Dall'altro lato, la valutazione **prestazionale** dei due modelli di Machine Learning (`ECGClassifier` e `PosturaClassifier`), condotta sui rispettivi dataset di addestramento tramite metriche standard di classificazione.

6.1 Setup di Test

La validazione funzionale è stata condotta utilizzando la catena hardware reale descritta nella Sezione 4 (si veda la Figura 4.1): il prototipo di board sensoristica IIT comunica via BLE con il dongle nRF52840 collegato al PC su cui gira l'applicazione *IIT BioDataAcq*. Su questo stesso PC sono stati avviati tutti i processi del backend (`docker-compose`, `fastapi_server.py`, `mqtt_subscriber.py`), mentre la dashboard medico è stata aperta in un browser separato per osservare in tempo reale l'effetto delle letture acquisite dal sensore.

Durante le sessioni di test, il paziente ha eseguito volontariamente sequenze di movimento controllate (a riposo, cammino, salita scale) per osservare la risposta del **PosturaClassifier** sulle diverse classi di attività motoria. Per il segnale ECG, la board sensoristica IIT offre la possibilità di iniettare un tracciato simulato: questa funzionalità è stata sfruttata per riprodurre un ritmo di fibrillazione atriale e verificare così la risposta dell'**ECGClassifier** a un'anomalia realistica, senza dover indurre l'aritmia sul paziente di test.

6.2 Validazione Funzionale End-to-End

La validazione funzionale ha avuto l'obiettivo di verificare che l'intero flusso dati attraversasse correttamente ogni componente del sistema, dal sensore fisico fino alla dashboard del medico e ritorno verso il paziente, utilizzando la catena hardware reale descritta nella Sezione 6.1 e non dati sintetici generati da `simulate_stream.py`.

6.2.1 Percorso Testato

Il test end-to-end ha ripercorso, in un'unica sessione continua, tutte le tappe della pipeline:

1. **Acquisizione:** il paziente ha indossato il prototipo di board sensoristica IIT, collegato via BLE al dongle nRF52840, e ha effettuato il login sull'app *IIT Bio-DataAcq* tramite il codice di accesso a 8 caratteri generato in precedenza dal medico dalla dashboard.
2. **Trasmissione MQTT:** avviata l'acquisizione, `mqtt_bridge.py` ha iniziato a pubblicare un messaggio al secondo su `smartcare/dati`, verificato osservando i log di `mqtt_subscriber.py` in ricezione.
3. **Classificazione e persistenza:** ogni messaggio è stato elaborato dai tre classificatori (Sezione 5.3) e salvato come annotazione su MongoDB. Per il segnale di movimento, il paziente ha eseguito sequenze controllate (fermo, salto, corsa) osservando il corrispondente cambio di etichetta di postura in tempo reale sulla dashboard.
4. **Rilevamento anomalia:** sfruttando la funzionalità di iniezione di un tracciato ECG simulato offerta dalla board sensoristica IIT, è stato riprodotto un episodio di fibrillazione atriale della durata di circa 20 secondi, per generare più letture anomale consecutive da raggruppare in un singolo episodio clinico.
5. **Notifica in tempo reale:** la dashboard ha ricevuto l'allarme via WebSocket entro 1–2 secondi dalla prima lettura anomala (toast, beep sonoro e, con permesso

attivo, notifica desktop nativa), mentre il badge sulla voce *Anomalie* della sidebar si è aggiornato coerentemente con il polling REST successivo.

6. **Validazione medica:** dalla sezione *Anomalie*, l'episodio è stato aperto nel modal di validazione; la traccia ECG estesa, inizialmente non disponibile, è diventata visualizzabile dopo pochi secondi tramite il pulsante **Riprova**. L'episodio è stato quindi validato come *vero positivo*, con nota clinica di accompagnamento.
7. **Propagazione al paziente:** contestualmente alla validazione, l'app paziente ha ricevuto sul topic `smartcare/validazioni` l'esito medico, mostrando un popup toast con beep e aggiornando il badge del pannello anomalie; aprendo il pannello, l'episodio validato è comparso con l'esito e la nota riportati correttamente.
8. **Storico:** infine, dalla sezione **Storico** della dashboard è stato possibile ritrovare lo stesso episodio, riaprirlo e verificarne la coerenza di dati ed esito rispetto a quanto validato in precedenza.

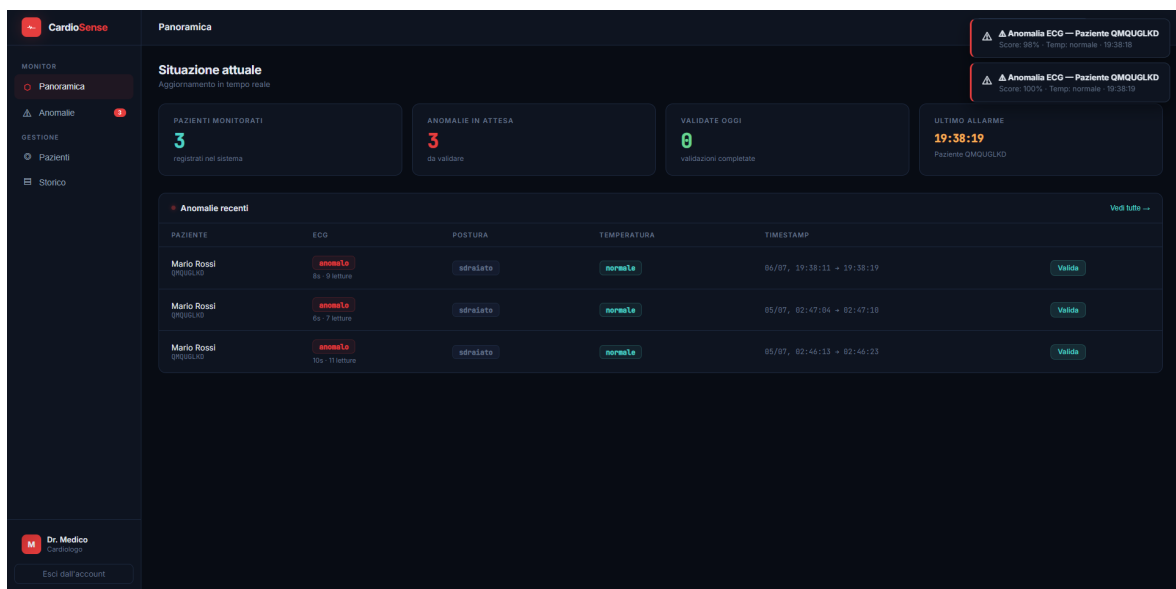


Figura 6.1: Dashboard medico al momento della ricezione dell'allarme.

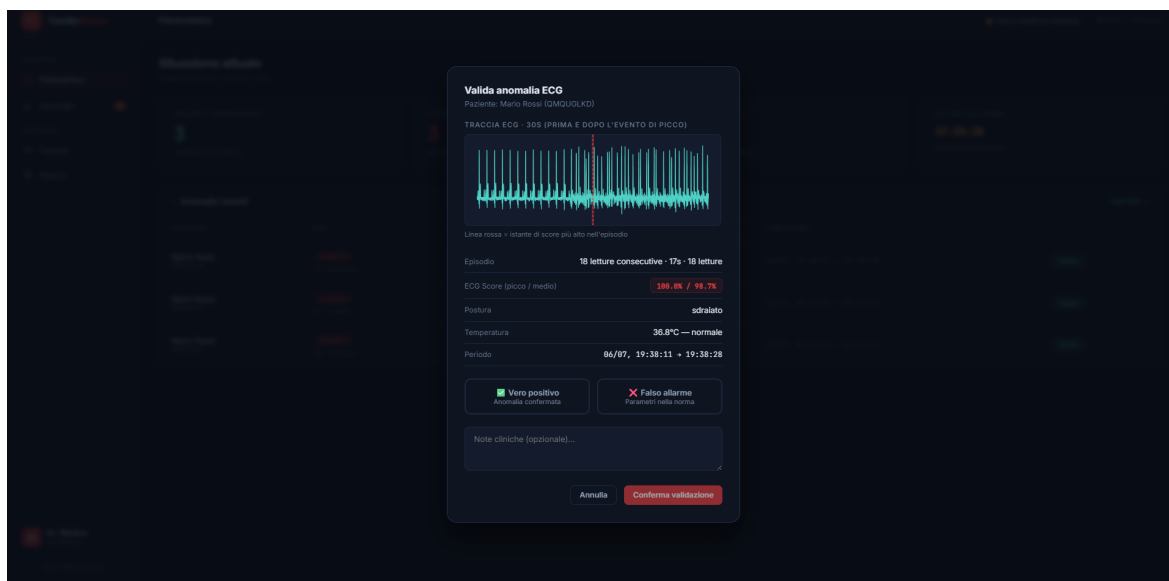


Figura 6.2: Modal di validazione con traccia ECG estesa e selezione dell'esito clinico (vero positivo / falso allarme).

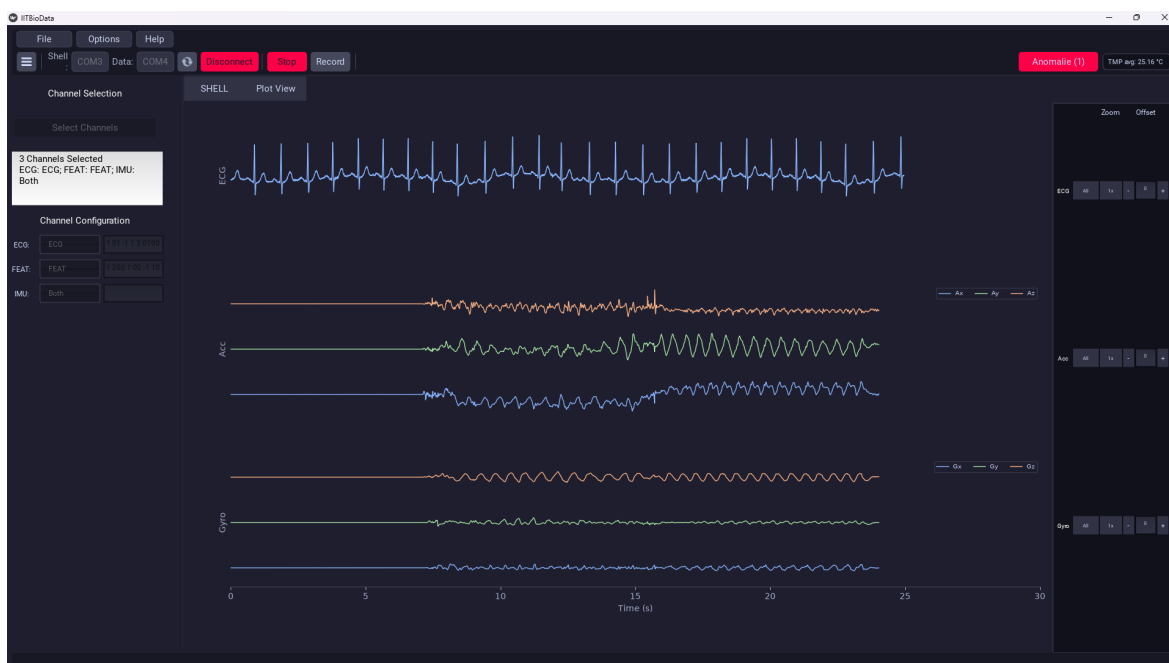


Figura 6.3: App paziente

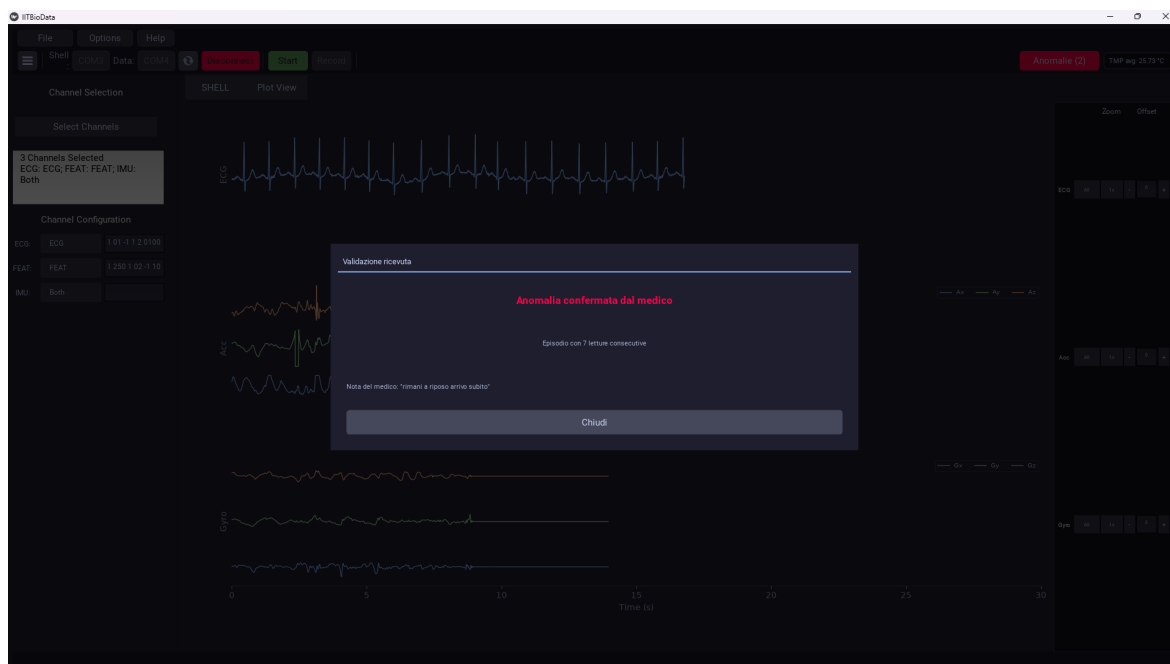


Figura 6.4: App paziente: badge anomalie aggiornato e popup con l’esito della validazione medica ricevuto in tempo reale.

6.2.2 Esito dei Test

La Tabella 6.1 riassume, per ciascuna funzionalità principale coinvolta nel percorso descritto, l’esito osservato durante la sessione di test.

Funzionalità testata	Esito
Login paziente tramite codice di accesso	Superato — codice validato dal server, sessione stabilita correttamente
Pubblicazione MQTT (ECG, IMU, temperatura)	Superato — un messaggio al secondo ricevuto senza perdite osservabili
Classificazione ECG	Superato — score coerente con il tracciato iniettato
Classificazione postura	Superato — etichetta aggiornata correttamente ad ogni cambio di attività
Classificazione temperatura	Superato — soglie cliniche applicate correttamente
Raggruppamento in episodio clinico	Superato — letture anomale consecutive raggruppate in un solo episodio

Funzionalità testata	Esito
Notifica in tempo reale (WebSocket + beep + notifica desktop)	Superato — allarme ricevuto entro 1–2s dalla prima lettura anomala
Traccia ECG estesa (asincrona)	Superato — inizialmente in elaborazione, disponibile dopo alcuni secondi tramite Riprova
Validazione medica dell’episodio	Superato — esito e nota salvati correttamente su MongoDB
Propagazione esito al paziente (topic validazioni)	Superato — popup e beep ricevuti lato app paziente in tempo reale
Consultazione storico e rivalutazione	Superato — episodio ritrovato con dati ed esito coerenti

Tabella 6.1: Esito dei test funzionali end-to-end.

Nel complesso, la sessione di test ha confermato il corretto funzionamento dell’intera catena, senza richiedere interventi manuali oltre a quelli previsti dal flusso applicativo (avvio dei servizi, login, validazione). Le uniche latenze percepibili — la disponibilità della traccia ECG estesa e l’aggiornamento del badge lato dashboard tramite polling REST — sono coerenti con i meccanismi asincroni descritti nelle Sezioni 5.2 e 5.8, e non hanno impedito una validazione clinica tempestiva dell’episodio.

6.3 Prestazioni dei Modelli di Machine Learning

Oltre alla validazione funzionale end-to-end (Sezione 6.2), i due classificatori basati su apprendimento supervisionato — `ECGClassifier` e `PosturaClassifier` — sono stati valutati singolarmente sui rispettivi dataset di addestramento, tramite uno split di test (20%, stratificato per classe) separato dal training set.

6.3.1 ECGClassifier — chfdb

Il modello è stato addestrato su 1 297 900 finestre di 10 intervalli R-R estratte da chfdb (su un totale di 1 622 375 finestre disponibili, di cui 236 310 anomale, pari a circa il 14,6% del dataset) e valutato su uno split di test di 324 475 finestre, con le classi bilanciate in fase di training tramite `class_weight='balanced'`.

Classe	Precision	Recall	F1-score	Support
Normale	0,99	0,98	0,98	277 213
Anomalo	0,87	0,93	0,90	47 262
Accuracy			0,97	324 475
Macro avg	0,93	0,95	0,94	324 475
Weighted avg	0,97	0,97	0,97	324 475

Tabella 6.2: Report di classificazione di `ECGClassifier` sullo split di test di `chfdb` (output di `classification_report`, script `train_ecg.py`).

6.3.2 PosturaClassifier — MHEALTH

Il modello è stato addestrato su 3882 finestre temporali estratte dal sottoinsieme filtrato di MHEALTH (le sette classi di attività motoria descritte in Sezione 5.3, dopo l'esclusione delle classi non rilevanti per il contesto clinico), di cui 3105 usate in training e 777 nello split di test, anch'esso con `class_weight='balanced'`. La distribuzione delle classi nel dataset completo è riportata nella colonna *Support* della Tabella 6.3 (valori riferiti al solo split di test).

Classe	Precision	Recall	F1-score	Support
in_piedi	0,99	0,99	0,99	122
seduto	1,00	0,98	0,99	123
sdraiato	0,99	1,00	1,00	123
camminata	0,98	0,98	0,98	123
salita_scale	0,96	0,96	0,96	122
corsa	1,00	0,99	1,00	123
salto	0,91	1,00	0,95	41
Accuracy			0,98	777
Macro avg	0,98	0,98	0,98	777
Weighted avg	0,98	0,98	0,98	777

Tabella 6.3: Report di classificazione di `PosturaClassifier` sullo split di test di MHEALTH (output di `classification_report`, script `train_postura.py`).

6.3.3 Discussione dei Risultati

I risultati ottenuti sui due dataset di riferimento permettono alcune osservazioni specifiche, coerenti con le scelte progettuali descritte nella Sezione 5.3.

ECGClassifier. La classe *Anomalo*, pur rappresentando solo il 14,6% circa del dataset *chfdb*, raggiunge un recall di 0,93 a fronte di una precision di 0,87: il modello privilegia quindi la sensibilità clinica, cioè la capacità di non lasciarsi sfuggire un'anomalia reale (recall alto), al costo di un numero relativamente più alto di falsi allarmi (precision più bassa). Questo comportamento è desiderabile nel contesto del sistema: un'anomalia non rilevata (falso negativo) comporterebbe un evento clinico mai notificato al medico, mentre un falso allarme (falso positivo) si traduce semplicemente in un episodio che il medico scarta in fase di validazione — un costo enormemente inferiore. L'accuracy complessiva del 97% è in parte trainata dalla classe maggioritaria *Normale* (recall 0,98), ma il *macro avg* (0,94 di F1-score) conferma che le prestazioni restano solide anche sulla classe minoritaria, grazie al bilanciamento delle classi in fase di training.

PosturaClassifier. Il modello raggiunge un'accuracy del 98% sul test set, con F1-score superiore a 0,95 su tutte le sette classi. La classe `salita_scale` è quella con le metriche più basse (precision e recall entrambe a 0,96), verosimilmente per la sua collocazione intermedia in termini di energia motoria tra *camminata* e le attività più intense (*corsa*, *salto*), il che la rende più soggetta a confusione con le classi limitrofe nella matrice di errore. La classe *salto*, pur essendo la meno rappresentata nel dataset (202 campioni totali, contro i circa 610–615 delle altre classi, per un support di soli 41 elementi nel test set), ottiene comunque un recall perfetto (1,00): il bilanciamento delle classi (`class_weight='balanced'`) sembra quindi compensare efficacemente la sotto-rappresentazione di questa attività, a fronte di una precision leggermente più bassa (0,91), cioè di un numero contenuto di finestre di altre classi erroneamente etichettate come *salto*.

6.4 Osservazioni su Latenza e Affidabilità

Oltre alla verifica funzionale del percorso completo (Sezione 6.2), la sessione di test con il dispositivo reale ha permesso di raccogliere alcune osservazioni qualitative sulla latenza percepita e sulla robustezza del sistema a condizioni di rete non ideali.

6.4.1 Latenza tra Evento Anomalo e Notifica

Come già descritto nella Sezione 5.8, il sistema espone due canali di aggiornamento con caratteristiche di latenza complementari:

- il canale **WebSocket** (`smartcare/allarmi` via `wss://`) ha mostrato, durante i test, una latenza percepita dell'ordine di massimo 1 secondo tra il momento in cui `mqtt_subscriber.py` classifica una lettura come anomala e la comparsa del

toast/beep sulla dashboard — un intervallo compatibile con la somma dei tempi di classificazione, scrittura su MongoDB, pubblicazione MQTT e rendering lato browser, senza colli di bottiglia percepibili in nessuno di questi passaggi;

- il canale di **polling REST** (`POLLING_EPISODI_MS = 8000`, Sezione 5.8) garantisce invece una convergenza dello stato mostrato entro al più 8 secondi, indipendentemente dall'esito del canale WebSocket. Nei test, il refresh anticipato innescato dalla ricezione di un allarme MQTT (`setTimeout(caricaEpisodi, 800)`) ha reso in pratica irrilevante l'attesa degli 8 secondi pieni nella quasi totalità dei casi, poiché il polling ordinario veniva quasi sempre anticipato dal trigger WebSocket.

La combinazione dei due canali si è quindi comportata secondo quanto previsto in fase di progettazione: il WebSocket fornisce la reattività immediata percepita dal medico, mentre il polling REST resta la rete di sicurezza che garantisce la convergenza dello stato anche in caso di perdita temporanea di messaggi MQTT o di allarmi generati mentre la dashboard non era ancora connessa al broker.

6.4.2 Osservazioni Pratiche col Dispositivo Reale

Testare il sistema con la catena hardware reale (board sensoristica IIT → dongle nRF52840 → app *IIT BioDataAcq*), anziché con i dati sintetici di `simulate_stream.py`, ha permesso di osservare alcuni aspetti non evidenti nei soli test con dati simulati:

- **Stabilità della connessione BLE:** nelle sessioni di test condotte, la connessione tra il prototipo di board sensoristica e il dongle nRF52840 non ha mostrato interruzioni spontanee per la durata delle prove effettuate; non sono quindi emerse evidenze dirette del comportamento del bridge MQTT in caso di perdita del collegamento BLE a monte, aspetto che meriterebbe un test dedicato e prolungato prima di un eventuale utilizzo continuativo del sistema.
- **Robustezza a disconnessioni del broker MQTT:** arrestando manualmente il container Mosquitto a dashboard aperta, l'indicatore di stato in `Topbar.jsx` è transitato correttamente da *connesso* a *disconnesso*, e poi a *in riconnessione* al riavvio del broker, senza necessità di ricaricare la pagina — confermando che il `reconnectPeriod: 3000` di `mqtt.js` (Sezione 5.8) gestisce autonomamente la riconnessione lato dashboard.
- **Asimmetria di riconnessione tra i client MQTT del sistema:** mentre la dashboard React e `mqtt_subscriber.py` (tramite la libreria MQTT sottostante) gestiscono esplicitamente la riconnessione automatica, il layer di integrazione lato paziente (`mqtt_bridge.py`, `patient_anomalies.py`) non implementa una logica

di retry dedicata in caso di disconnessione dal broker: un'interruzione prolungata della connessione di rete sul lato paziente richiederebbe quindi, allo stato attuale, un riavvio dell'applicazione per ripristinare la pubblicazione/sottoscrizione MQTT. Si tratta di un'area da rafforzare in vista di un utilizzo prolungato del sistema al di fuori di un contesto di test controllato.

- **Nessuna perdita di messaggi osservata a QoS 1:** l'uso sistematico di qos=1 su tutte le pubblicazioni MQTT del sistema (dati, allarmi, validazioni) non ha mostrato, nelle sessioni di test condotte su rete locale, perdite di messaggi né duplicazioni percepibili lato dashboard o app paziente.

Nel complesso, le osservazioni raccolte confermano la solidità dell'architettura a doppio canale (MQTT + REST) discussa nella Sezione 5.8 per quanto riguarda i client dotati di riconnessione automatica, mentre evidenziano nella riconnessione MQTT lato paziente un punto di attenzione da annoverare, insieme a quelli già discussi nella Sezione 5.10.2, tra i possibili sviluppi futuri del sistema in vista di un impiego continuativo al di fuori del contesto di test.

Capitolo 7

Conclusioni e Sviluppi Futuri

7.1 Sintesi del Lavoro Svolto

Il progetto *SmartCare* ha realizzato un sistema IoT closed-loop per il monitoraggio domiciliare di pazienti con scompenso cardiaco, integrando in un'unica architettura end-to-end l'acquisizione multiparametrica dei segnali fisiologici (ECG, movimento e postura tramite IMU a 6 assi, temperatura corporea), la loro classificazione automatica tramite modelli di apprendimento supervisionato, un meccanismo di annotazione clinica che coinvolge attivamente il medico nella validazione degli eventi rilevati, e un ciclo di ri-addestramento periodico che chiude il loop tra intelligenza artificiale e giudizio clinico.

Il lavoro ha richiesto di affrontare problematiche di natura eterogenea: dall'integrazione non invasiva con un'applicazione di acquisizione preesistente di proprietà di IIT (*IIT BioDataAcq*), alla progettazione di un'architettura event-driven basata su MQTT capace di notificare in tempo reale sia il medico sia il paziente, fino alla gestione della coerenza fisica delle unità di misura tra dataset di training (MHEALTH) e sensore reale.

La validazione condotta nel Capitolo 6 ha confermato, sia sul piano funzionale sia su quello prestazionale, che il sistema realizzato soddisfa i requisiti definiti nel Capitolo 2: il percorso end-to-end funziona correttamente con il dispositivo wearable reale (Sezione 6.2), i due classificatori raggiungono metriche solide sui rispettivi dataset di riferimento (Sezione 6.3), e l'architettura a doppio canale di notifica si comporta secondo quanto previsto in termini di latenza e robustezza (Sezione 6.4).

7.2 Limiti dell'Implementazione Attuale

Accanto ai risultati positivi, il lavoro svolto ha messo in luce una serie di limiti. Di seguito sono raccolti e discussi criticamente, organizzati per area tematica.

7.2.1 Modelli di Machine Learning

- **Soglia di classificazione ECG fissa:** la soglia (0,5) utilizzata da `ECGClassifier` per mappare la probabilità restituita dal Random Forest in un'etichetta binaria è stata scelta empiricamente e non calibrata tramite tecniche dedicate. I risultati della Sezione 6.3.1 mostrano un buon compromesso tra precision e recall sulla classe *Anomalo*, ma un affinamento della soglia — eventualmente specifico per singolo paziente, sulla base della sua storia clinica — potrebbe ridurre ulteriormente il numero di falsi allarmi senza compromettere la sensibilità.
- **Retraining incrementale ECG dipendente dalla cache chfdb:** il meccanismo descritto in Sezione 5.6 salta in modo sicuro il ciclo notturno se la cache locale delle feature chfdb non è disponibile e PhysioNet non è raggiungibile, ma questo comportamento — pur preferibile a un retrain instabile — comporta la perdita silenziosa di un ciclo di aggiornamento; un meccanismo di allerta esplicito verso l'amministratore di sistema in caso di retrain saltato ripetutamente costituirebbe un miglioramento naturale.

7.2.2 Architettura e Gestione dello Stato

- **Assenza di cleanup automatico dei buffer per paziente:** come discusso in Sezione 5.10.2, `PosturaClassifier` espone un metodo `dimentica_paziente()` mai invocato automaticamente da nessun chiamante; lo stato interno di ogni paziente monitorato resta quindi in memoria per l'intera vita del processo `mqtt_subscriber.py`, anche dopo la sua disconnessione. Con il numero limitato di pazienti utilizzato nei test, questo non ha causato problemi osservabili, ma un deployment con una popolazione di pazienti più ampia e variabile nel tempo richiederebbe un meccanismo di cleanup su timeout di inattività.
- **Asimmetria nel badge anomalie lato paziente:** il contatore di anomalie non viste dell'app paziente (`AnomaliesMonitor`) è mantenuto in-memory e viene azzerato al riavvio dell'applicazione, mentre il badge lato medico è calcolato da query persistenti su MongoDB. Lo storico completo resta sempre accessibile e corretto tramite il popup `AnomaliesPopup`, ma la discrepanza tra i due contatori potrebbe generare una percezione di incoerenza per un utente che riavvii frequentemente l'app.

- **Riconnessione MQTT non gestita esplicitamente lato paziente:** come emerso nella Sezione 6.4, a differenza della dashboard React e di `mqtt_subscriber`, il layer di integrazione lato paziente (`mqtt_bridge.py`, `patient_anomalies.py`) non implementa una logica di retry dedicata in caso di disconnessione dal broker. Un'interruzione di rete prolungata richiederebbe, allo stato attuale, il riavvio dell'applicazione paziente per ripristinare la corretta pubblicazione e sottoscrizione MQTT.

7.2.3 Sicurezza e Portabilità

- **Portabilità dei certificati TLS:** come discusso nella Sezione 5.10.1, la CA generata da `mkcert` è di fiducia solo sulla macchina su cui è stata installata. Questo è accettabile in un contesto di sviluppo e demo locale, ma un deployment distribuito (più medici, più postazioni, più pazienti su reti diverse) richiederebbe una CA condivisa o certificati firmati da un'autorità di certificazione riconosciuta.
- **CORS in sviluppo:** `allow_origins` in `fastapi_server.py` è attualmente configurato per accettare l'origine locale del dev server Vite; prima di un eventuale deployment pubblico andrebbe ristretto esplicitamente al dominio di produzione della dashboard, evitando in ogni caso la combinazione di un wildcard (*) con `allow_credentials=True`.
- **Assenza di mutua autenticazione TLS (mTLS):** come osservato in Sezione 5.10.1, l'architettura implementa TLS solo lato server; i client non possiedono un proprio certificato, per cui l'identità di dispositivo e app paziente non è verificata crittograficamente dal broker, ma si affida al solo codice di accesso alfanumerico.

7.3 Sviluppi Futuri

A partire dai limiti discussi nella sezione precedente, è possibile delineare alcune direzioni di sviluppo che estenderebbero significativamente le capacità e la robustezza del sistema.

7.3.1 Estensione dei Parametri Fisiologici Monitorati

Il sistema è stato progettato, secondo il principio di manutenibilità discusso nei requisiti non funzionali (Sezione 2), in modo da poter accogliere nuove tipologie di sensore o nuovi classificatori senza modifiche strutturali al core: l'interfaccia comune `BaseClassifier` (pattern Strategy, Sezione 5.3) rende l'aggiunta di un nuovo classificatore un'estensione piuttosto che una modifica invasiva. Uno sviluppo naturale

consisterebbe nell'integrare ulteriori parametri clinicamente rilevanti per lo scompenso cardiaco — ad esempio la saturazione periferica di ossigeno (SpO_2) — qualora il prototipo hardware IIT venga esteso con i sensori corrispondenti.

7.3.2 Interoperabilità Clinica: HL7 FHIR

Come anticipato nell'introduzione (Sezione 2), il sistema attuale adotta uno schema dati proprio (documenti `Annotation` su MongoDB, modelli `SQLAlchemy` su MySQL) non conforme a standard di interoperabilità clinica. Un'evoluzione significativa in vista di un utilizzo reale in ambito ospedaliero sarebbe l'adozione dello standard **HL7 FHIR**, mappando le annotazioni verso risorse `Observation` e le validazioni mediche verso risorse `ServiceRequest` o `DiagnosticReport`, rendendo il sistema potenzialmente integrabile con cartelle cliniche elettroniche esistenti.

7.3.3 Robustezza della Connettività Lato Paziente

Come discusso in Sezione 7.2, il layer di integrazione lato paziente non implementa attualmente una logica di riconnessione esplicita in caso di caduta della connessione MQTT, a differenza degli altri client del sistema. L'introduzione di un meccanismo di retry con backoff, analogo al `reconnectPeriod` già impiegato da `mqtt.js` nella dashboard, renderebbe l'esperienza dell'app paziente più resiliente a interruzioni di rete transitorie, senza richiedere il riavvio manuale dell'applicazione.

7.3.4 Gestione del Ciclo di Vita dello Stato per Paziente

L'introduzione di un meccanismo di cleanup automatico dei buffer per paziente — invocando `dimentica_paziente()` su un timeout di inattività configurabile, o alla ricezione di un evento esplicito di disconnessione — estenderebbe il pattern di isolamento per paziente già discusso in Sezione 5.10.2, rendendo il sistema scalabile anche a deployment con una popolazione di pazienti ampia e variabile nel tempo, senza una crescita indefinita dello stato mantenuto in memoria dai processi di lunga durata.

7.3.5 Hardening della Sicurezza per il Deployment in Produzione

In vista di un eventuale utilizzo clinico reale — per il quale, si ribadisce, il sistema attuale resta un prototipo dimostrativo non certificato come dispositivo medico — sarebbe necessario un percorso di hardening della sicurezza che includa: l'adozione di certificati TLS firmati da un'autorità di certificazione riconosciuta, in sostituzione della CA locale `mkcert`; l'introduzione di mutua autenticazione TLS (mTLS) tra i client

e il broker Mosquitto; la restrizione esplicita della configurazione CORS al dominio di produzione della dashboard; e una revisione della gestione dei segreti (chiave JWT, credenziali dei database) attualmente mantenuti in file `.env` locali, in favore di un sistema di secret management dedicato. Un percorso di conformità formale alle norme MDR (EU 2017/745) per i dispositivi medici, già richiamato tra i requisiti non funzionali (Sezione 2), rimarrebbe comunque un prerequisito imprescindibile per qualunque utilizzo del sistema al di fuori del contesto dimostrativo e accademico in cui è stato sviluppato.

Riferimenti Bibliografici

- [1] Humanitas Research Hospital. *Che cos'è lo scompenso cardiaco e come si riconosce?* <https://www.humanitas.it/news/cose-lo-scompenso-cardiaco-si-riconosce/>. Ultimo aggiornamento: maggio 2023. Accesso: giugno 2026. 2020.
- [2] Gianluigi Savarese, Peter Michael Becher, Lars H. Lund, Petar Seferovic, Giuseppe M.C. Rosano e Andrew J.S. Coats. «Global burden of heart failure: a comprehensive and updated review of epidemiology». In: *Cardiovascular Research* 118.17 (2022). Disponibile su: <https://academic.oup.com/cardiovascres>. DOI: [10.1093/cvr/cvac013](https://doi.org/10.1093/cvr/cvac013).
- [3] HL7 International. *HL7 FHIR Release 5 – Fast Healthcare Interoperability Resources*. <https://www.hl7.org/fhir/>. Versione corrente: R5 (v5.0.0), pubblicata il 26 marzo 2023. 2023.
- [4] Karim Bayoumy et al. «Smart wearable devices in cardiovascular care: where we are and how to move forward». In: *Nature Reviews Cardiology* 18.8 (2021), pp. 581–599. DOI: [10.1038/s41569-021-00522-7](https://doi.org/10.1038/s41569-021-00522-7).
- [5] Andrea Rucco, Angela Tafadzwa Shumba, Teodoro Montanaro, Ilaria Sergi e Luigi Patrono. «Enhancing Chronic Heart Failure Monitoring, Prevention, and Management With IoT and AI: A Systematic Literature Review». In: *IEEE Journal of Biomedical and Health Informatics* 30.5 (2026), pp. 3768–3783. DOI: [10.1109/JBHI.2025.3628501](https://doi.org/10.1109/JBHI.2025.3628501).
- [6] N. Vithyalakshmi, Sharada Prasad N, Mahesh Kumar A. S, Madhu B K, Vipul Vekariya e Gitanjali Shrivastava. «Wearable IoT-based Health Monitoring System with AI-Driven Alerts for Real-Time Patient Anomaly Detection». In: *2025 3rd International Conference on Data Science and Information System (ICDSIS)*. 2025, pp. 1–5. DOI: [10.1109/ICDSIS65355.2025.11071072](https://doi.org/10.1109/ICDSIS65355.2025.11071072).
- [7] Safia Naveed.S, Nabeena Ameen e Nulin Jeriba J. «Enhanced IoT System for Real-Time Detection and Monitoring of Cardiac Events Using Machine Learning Algorithms». In: *2024 3rd Edition of IEEE Delhi Section Flagship Conference (DELCON)*. 2024, pp. 1–6. DOI: [10.1109/DELCON64804.2024.10866172](https://doi.org/10.1109/DELCON64804.2024.10866172).
- [8] Naman Tiwari, Vipin Kr. Kushwaha, Satya Prakash Yadav, Shiva Gupta e B. Sharan. «Wearable IoT Devices for Real-Time Health Monitoring and Anomaly

- Detection». In: *2025 Modern Electronics Devices and Intelligent Communication Systems (MEDCOM)*. 2025, pp. 320–325. DOI: [10.1109/MEDCOM67532.2025.11404931](https://doi.org/10.1109/MEDCOM67532.2025.11404931).
- [9] N. Hinrichs et al. «Artificial intelligence based real-time prediction of imminent heart failure hospitalisation in patients undergoing non-invasive telemedicine». In: *Frontiers in Cardiovascular Medicine* 11 (2024), p. 1457995. DOI: [10.3389/fcvm.2024.1457995](https://doi.org/10.3389/fcvm.2024.1457995).
- [10] Nitin Naik. «Choice of Effective Messaging Protocols for IoT Systems: MQTT, CoAP, AMQP and HTTP». In: *2017 IEEE International Systems Engineering Symposium (ISSE)*. 2017, pp. 1–7. DOI: [10.1109/SysEng.2017.8088251](https://doi.org/10.1109/SysEng.2017.8088251).
- [11] FiloSottile. *mkcert: A simple zero-config tool to make locally trusted development certificates*. <https://github.com/FiloSottile/mkcert>. 2024.
- [12] Eclipse Foundation. *Eclipse Mosquitto – An open source MQTT broker*. <https://mosquitto.org>. 2024.
- [13] Sebastián Ramírez. *FastAPI – Framework, high performance, easy to learn, fast to code, ready for production*. <https://fastapi.tiangolo.com>. 2024.
- [14] Leo Breiman. «Random Forests». In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [15] Donald S. Baim, Wilson S. Colucci, E. Scott Monrad, Harold S. Smith, Robert F. Wright, Ann Lanoue, David F. Gauthier, Bernard J. Ransil, William Grossman e Eugene Braunwald. «Survival of patients with severe congestive heart failure treated with oral milrinone». In: *Journal of the American College of Cardiology* 7.3 (1986). Dataset disponibile su: <https://physionet.org/content/chfdb/1.0.0/>, pp. 661–670. DOI: [10.1016/S0735-1097\(86\)80478-8](https://doi.org/10.1016/S0735-1097(86)80478-8).
- [16] Ary L. Goldberger, Luis A.N. Amaral, Leon Glass, Jeffrey M. Hausdorff, Plamen Ch. Ivanov, Roger G. Mark, Joseph E. Mietus, George B. Moody, Chung-Kang Peng e H. Eugene Stanley. «PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals». In: *Circulation* 101.23 (2000). Dataset CHF: <https://physionet.org/content/chfdb/e215-e220>. DOI: [10.1161/01.CIR.101.23.e215](https://doi.org/10.1161/01.CIR.101.23.e215).
- [17] Oresti Banos, Rafael Garcia e Alejandro Saez. *MHEALTH*. UCI Machine Learning Repository. 2014. DOI: [10.24432/C5TW22](https://doi.org/10.24432/C5TW22).
- [18] Fabian Pedregosa et al. «Scikit-learn: Machine Learning in Python». In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830. URL: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [19] Chen Xie, Lucas McCullum, Alistair Johnson, Tom Pollard, Brian Gow e Benjamin Moody. «Waveform Database Software Package (WFDB) for Python». In: *PhysioNet* (gen. 2023). Version 4.1.0. DOI: [10.13026/9njx-6322](https://doi.org/10.13026/9njx-6322). URL: <https://doi.org/10.13026/9njx-6322>.

- [20] The Kivy Authors. *Kivy Documentation*. Version 2.3.1, Accessed: 9 luglio 2026. Kivy Foundation. 2024. URL: <https://kivy.org/doc/stable/>.
- [21] Meta Platforms, Inc. *React — A JavaScript library for building user interfaces*. <https://react.dev/>. Documentazione ufficiale. 2024.
- [22] Even You. *Vite — Next Generation Frontend Tooling*. <https://vitejs.dev/>. Documentazione ufficiale. 2024.
- [23] Matteo Collina. *MQTT.js — A client library for the MQTT protocol, written in JavaScript*. <https://www.npmjs.com/package/mqtt>. Pacchetto npm. 2024.
- [24] Docker Inc. *Docker — Accelerated Container Application Development*. <https://www.docker.com>. 2024.
- [25] makerdiary. *nRF52840 MDK USB Dongle*. Documentazione tecnica: <https://wiki.makerdiary.com/nrf52840-mdk-usb-dongle/>. 2024.